

Cursor Training Program

AI-Assisted Development with Cursor

Springpeople · 2-day instructor-led course · Modules 1–10

Course Agenda

Module	Title	Day	Duration
1	Mental Models for AI-Assisted Development	Day 1	~60 min
2	Cursor Editor Essentials	Day 1	~90 min
3	Agent Modes and Tools	Day 1	~60 min
4	Customizing Cursor for Your Team	Day 1	~60 min
5	Cursor CLI and Local Automation	Day 1	~60 min
6	Cloud Agents in the UI	Day 2	~90 min
7	Cursor API Foundations	Day 2	~60 min
8	Cloud Agents API and Webhooks	Day 2	~60 min
9	Admin and Analytics APIs	Day 2	~75 min
10	AI Code Tracking and Reporting	Day 2	~20 min

Total: ~11.5 hours across 2 days (hands-on labs + demonstrations)

Day 1 — Foundations & Editor Workflows

Module	Title	Focus	Duration
1	Mental Models for AI-Assisted Development	Foundations	~60 min
2	Cursor Editor Essentials	Hands-On	~90 min
3	Agent Modes and Tools	Hands-On + Concept	~60 min
4	Customizing Cursor for Your Team	Hands-On + Walkthrough	~60 min
5	Cursor CLI and Local Automation	Hands-On	~60 min

Concept blocks, hands-on exercises, team customization, and CLI automation.

Day 2 — Cloud Agents, APIs & Analytics

Module	Title	Focus	Duration
6	Cloud Agents in the UI	Hands-On + Demonstration	~90 min
7	Cursor API Foundations	Concept + Hands-On	~60 min
8	Cloud Agents API and Webhooks	Hands-On	~60 min
9	Admin and Analytics APIs	Hands-On + Demonstrations	~75 min
10	AI Code Tracking and Reporting	Hands-On + Take-Home	~20 min

Cloud agents, programmatic APIs, admin analytics, and AI code tracking.

Mental Models for AI-Assisted Development

Module 1 · Day 1 (Foundations)

Cursor Training Program · Concept block · ~60 min

Module Overview

Aspect	Details
Duration	~60 minutes
Format	Concept block (foundational theory)
Prerequisites	None – this is the starting point
Module Goal	Build accurate mental models of how AI coding assistants work, their limitations, and how to use them effectively

Learning Objectives

By the end of this module, participants will be able to:

- Explain why AI outputs are probabilistic, not deterministic
- Identify and mitigate hallucinations in coding contexts
- Understand token-based pricing and cost optimization
- Master context as the single most valuable AI skill
- Distinguish between tool calling, MCP, and autonomous agents
- Define the developer's evolving role with AI agents

Lesson 1.1

How AI Models Work

Concept · 12 minutes

Why Outputs Are Probabilistic

Unlike traditional software that gives the same output for the same input, AI models generate responses based on **probability distributions**.

At its simplest, an LLM is a **next-token prediction engine**.

Given a sequence of tokens, it predicts what comes next — then samples, appends, repeats.

Next-Token Prediction

```
Input: "def calculate_sum(a, b):"
Model thinks: "What's most likely next?"

Probabilities:
- "return" → 85%
- "print"  → 8%
- "pass"   → 4%
- "for"    → 2%
- Other    → 1%
```

Traditional Code vs. AI Model

Traditional Code	AI Model
Deterministic (same input → same output)	Probabilistic (different outputs possible)
You control the logic	You influence, but don't control
Errors are bugs	Errors are features of probability
Predictable behavior	Needs management via parameters

Implication: Never trust a single run as ground truth.

What Determines AI Output?

Factors That Shape Output	
1. Training Data	→ What the model "learned"
2. Prompt	→ Your instruction (most influential)
3. Temperature	→ Randomness level (0 = deterministic)
4. Top-p / Top-k	→ Token selection pool size

- | | |
|-----------------------|--------------------------------------|
| 5. System Prompt | → Persistent behavioral guidelines |
| 6. Context Window | → What the model can "see" |
| 7. Model Architecture | → Different models, different biases |

Key Parameters You Control

Parameter	What It Does	Best For
Temperature	Randomness (0 = deterministic, 1 = creative)	Bug fixes (low), brainstorming (high)
Top-p	Nucleus sampling – limits token pool	Balanced responses
Max Tokens	Limits response length	Controlling cost

```
temperature: 0.2    # focused
top_p: 0.9          # balanced
max_tokens: 4000    # cap length
```

Temperature Impact

Same prompt: *"Write a function to reverse a string"*

```
# Temperature 0.1 – very deterministic
def reverse_string(s):
    return s[::-1]

# Temperature 0.7 – balanced (adds edge cases)
def reverse_string(s):
    if not s: return s
    return s[::-1]

# Temperature 1.2 – creative, potentially unstable
def flip_the_text(text): ...
```

The Training Gap

Models are frozen at their training cutoff date. They don't know:

- Code written after their training date
- Your company's internal APIs
- Your specific architecture decisions
- Recent library updates (unless in context)

Implication: You must provide this information in the prompt or context.

Lesson 1.2

Hallucinations

Concept · 10 minutes

What Are Hallucinations?

Confident-sounding outputs that are **factually wrong**, made up, or don't exist.

Most dangerous form: the model sounds **completely confident** while being **completely wrong**.

Hallucinations in Code

Type	Example	How to Spot
Fake APIs	<code>import nonexistent_library</code>	Check docs; import fails
Wrong parameters	Incorrect function signature	Type checking
Invented methods	<code>list.reverse_in_place()</code>	Know the standard library
Confident nonsense	"This is the standard way to..."	Cross-reference
Outdated syntax	Old Python 2 style	Know version differences

Why Models Hallucinate

Root Causes of Hallucination

Training Data Gaps	→ Model "guesses" when uncertain
Probability Pressure	→ Must output SOMETHING
Pattern Overfitting	→ Sees patterns that don't exist
No True/False Circuit	→ Models don't have truth checking
Confidence Calibration	→ Sounds confident when wrong

Example: Confident Wrong

User: "How do I use requests for async calls?"

```
# Hallucinated (confident, wrong)
import requests.async as async_requests
```

```
response = await async_requests.get('https://api.example.com')

# Reality: requests does NOT have async support.
# Correct answer: Use httpx or aiohttp
```

Hallucination Mitigation Strategies

Strategy	How It Works	Example
Grounding	Provide source material	Paste library docs into context
Verification	Ask for citations	"Which line of the docs shows this?"
Constrained decoding	Limit possible outputs	JSON mode, regex patterns
Self-consistency	Ask multiple times, compare	Run same prompt 3×, take majority
Low temperature	Reduce randomness	temperature: 0.1
Tool use	Let model search/lookup	Enable web search for docs

Hallucination Detection Checklist

Before accepting AI-generated code, verify:

- Do the imported libraries exist?
- Are function signatures correct?
- Does the syntax match my language version?
- Are there obvious logic errors?
- Would this code actually run?
- Does the model cite sources you can verify?

The Developer's Mindset

"Trust, but verify – especially when the AI sounds most confident."

- Hallucinations decrease with better prompts and context
- They never fully disappear
- You are the human-in-the-loop responsible for verification
- Experience helps you "smell" potential hallucinations

Lesson 1.3

Tokens and Pricing

Concept · 10 minutes

What Is a Token?

A token is the atomic unit of processing for LLMs — not a word, not a character.

Language	Example	Token Count
English	"Hello world"	2 tokens (~0.75 words/token)
English	"Congratulations"	1 token
Code	<code>function calculateTotal()</code>	~5 tokens (~2–4 chars/token)
Chinese	"你好世界"	4–8 tokens

Why tokens matter: You pay per token · Context windows are measured in tokens · Token limits determine how much code the AI can "see"

Input vs. Output Pricing

Input tokens (prompt, code context, retrieved docs) cost **less** than **output tokens** (generated code and explanations).

Output is often **5–8× more expensive** — generation is more compute-intensive than reading.

Model Pricing Examples

Model	Input (per 1M)	Output (per 1M)	Output/Input
GPT-5 Mini	\$0.25	\$2.00	8×
Claude 4.5 Haiku	\$1.00	\$5.00	5×
GPT-5.3 Codex	\$1.75	\$14.00	8×
Gemini 3.1 Pro	\$2.00	\$12.00	6×
Claude 4.6 Sonnet	\$3.00	\$15.00	5×
Claude 4.7 Opus	\$5.00	\$25.00	5×
GPT-5.5	\$5.00	\$30.00	6×

What 1 Million Tokens Looks Like

Content Type	Approximate Amount
Plain English text	~750,000 words (~1,500 pages)
Python code	~250,000–500,000 lines
Average conversation	5–10 sessions
Full codebase	Small to medium project

Cost Calculation Example

```
prompt_tokens = 5000      # instructions + context
output_tokens = 2000      # AI response

model = "claude-4.6-sonnet"
input_price  = 3.00       # per 1M tokens
output_price = 15.00      # per 1M tokens

input_cost   = (5000 / 1_000_000) * 3.00
output_cost  = (2000 / 1_000_000) * 15.00
total_cost   = input_cost + output_cost    # ~$0.045 (4.5 cents)
```

Cost Optimization Strategies

Strategy	How It Works	Impact
Use cheaper models	Mini/Haiku for simple tasks	5–20× reduction
Reduce context	Only send relevant code	2–5× reduction
Cache responses	Reuse common answers	Variable
Batch operations	Combine multiple tasks	30–50% reduction
Monitor usage	Track spending per user	Prevents surprises
Set limits	Monthly spending caps	Budget protection

Real-World Cost Bounds

Usage Level	Monthly Cost	What You Can Do
Light	\$10–20	Occasional questions, small fixes
Medium	\$50–100	Daily coding, regular agent use
Heavy	\$200–500	Full-time AI assistance, multiple agents
Enterprise	\$1000+	Team usage, automation, CI/CD

The Cache Effect

Models can cache frequently used content:

- **Cache Write:** Cost to initially store
- **Cache Read:** **Much cheaper** than fresh input (80–95% savings)


```
# First request → pays full input price
# Second request → same context → pays cache read price
```

Context discipline = cost discipline.

Lesson 1.4

Context

Concept · 12 minutes · The single most valuable AI skill

What Is Context?

Context = all the information the model has access to when generating a response.

What Goes Into Context

System Prompt

→ "You are a helpful coding assistant"

User Prompt

→ "Fix this bug: ..."

Code Files

→ Current file, related files

Conversation

→ Previous exchanges

Retrieved Docs

→ Library documentation, examples

Tool Outputs

→ Results from search, file reads

Constraints

→ "Only use standard library"

The Context Window Limit

Model	Context Window	Pages of Text	Lines of Code
Claude 4 (Haiku / Sonnet / Opus)	200k	~150	~50,000
GPT-5 Mini / GPT-5.3 Codex	272k	~200	~70,000

When you exceed context: Oldest content gets truncated · Critical information may be dropped

Context engineering = knowing what to put in, what to leave out, and how to structure it.

Context Checklist

Before every AI interaction, ask:

- What problem am I trying to solve?

- What files/code does the model need to see?
- What would a human need to know to help me?
- What information can I safely leave out?
- Is my context under the token limit?
- Have I included relevant error messages?
- Have I specified constraints (libraries, version, style)?

Good vs. Bad Context

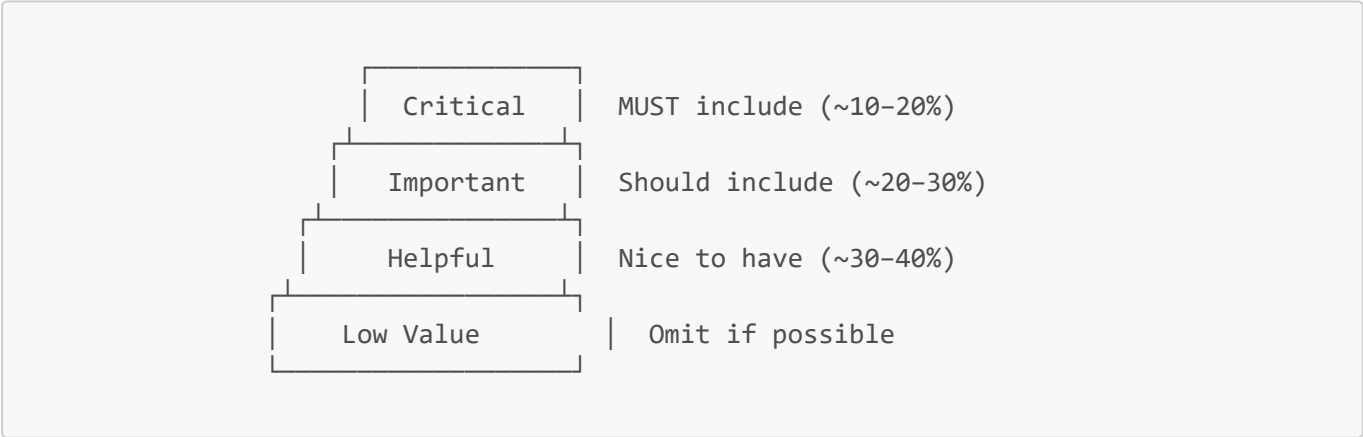
BAD (vague):

"Fix this bug: my code doesn't work"

GOOD (specific):

Python function sorts dicts by key but raises KeyError.
Code: def sort_by_key(data, key): ...
Input: [{'name': 'Alice'}, {'age': 30}]
Using Python 3.11. Expected: skip dicts without the key.

Context Prioritization Pyramid



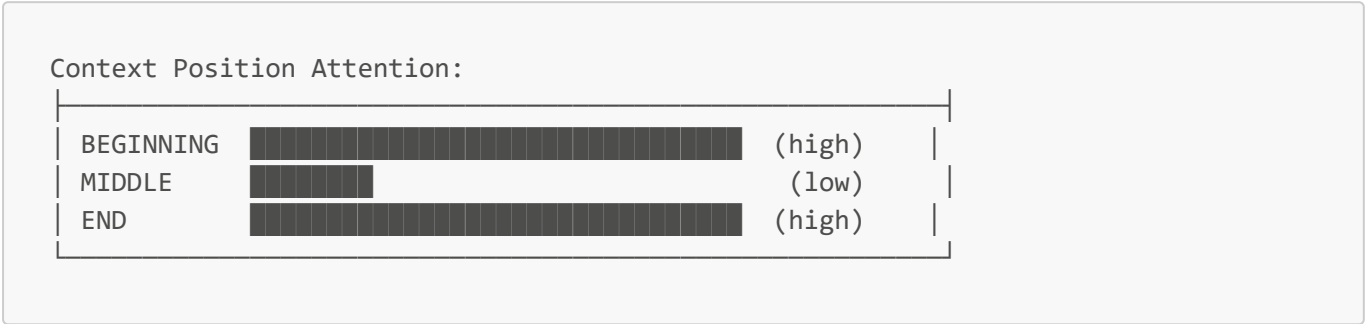
Context Window Management

Strategy	How It Works	When to Use
Summarization	Compress earlier conversation	Long sessions
Selective inclusion	Only relevant files	Large codebases
Chunking	Split across multiple calls	Exceeding limit
Hierarchical	Summaries + details on demand	Complex projects

Strategy	How It Works	When to Use
Vector retrieval	Semantic search for relevant context	Very large codebases

The "Lost in the Middle" Problem

Models pay **most attention to the beginning and end** of context, and **less to the middle**.



Implication: Put critical information at the beginning OR end, not the middle.

Lesson 1.5

Tool Calling and MCP

Concept · 8 minutes

What Is Tool Calling?

Tool calling (function calling) lets the AI request execution of external functions.

The AI **doesn't execute code** — it outputs a structured request that **your system** executes.

```
User: "What's the weather in Tokyo?"
→ AI decides: need weather data
→ AI outputs: tool_call { name: "get_weather", args: { city: "Tokyo" } }
→ Your system executes get_weather("Tokyo")
→ Result returned to AI
→ AI: "The weather in Tokyo is 22°C and sunny"
```

Common Tool Types in Development

Tool	Purpose	Example
read_file	Read code files	"Show me the auth module"
edit_file	Modify code	"Add error handling to line 42"

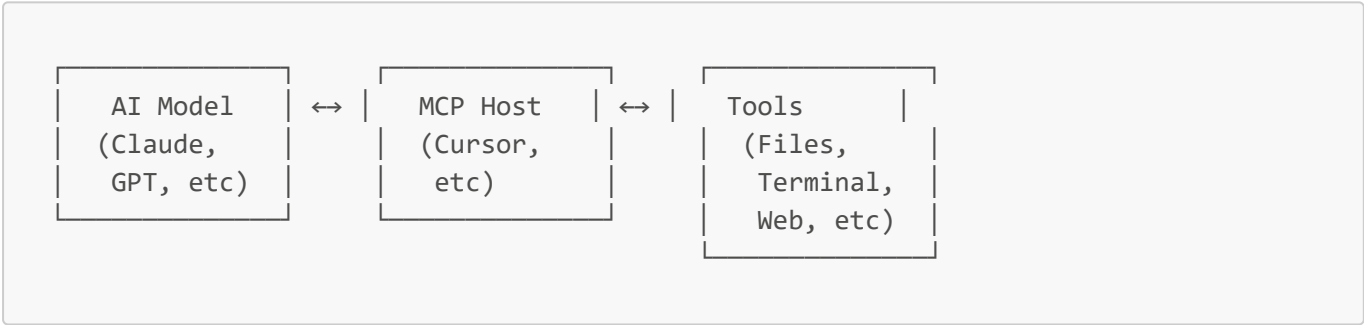
Tool	Purpose	Example
search_code	Find patterns	"Find all uses of this function"
run_terminal	Execute commands	"Run the tests"
web_search	Find documentation	"Look up pandas DataFrame API"
browser	Browse web pages	"Open the PR and review it"
git	Version control	"Create a branch and commit"

MCP (Model Context Protocol)

"USB-C for AI — one protocol that works across different tools."

Without MCP: Each tool needs custom integration

With MCP: Tools advertise their capabilities; AI discovers them dynamically



Why MCP Matters

Benefit	Explanation
Interoperability	Same tools work across different AI models
Discoverability	AI can learn what tools are available
Standardization	One protocol, not dozens of custom APIs
Extensibility	Add new tools without changing AI logic

Tool Calling Best Practices

1. **Define clear tool schemas** — name, description, parameters
2. **Validate tool calls** before execution — allowlist + parameter checks
3. **Set timeouts** — e.g., 30 seconds max per tool
4. **Log all tool calls** — audit trail for debugging
5. **Require human approval** for destructive actions — never auto-run writes/deletes

Lesson 1.6

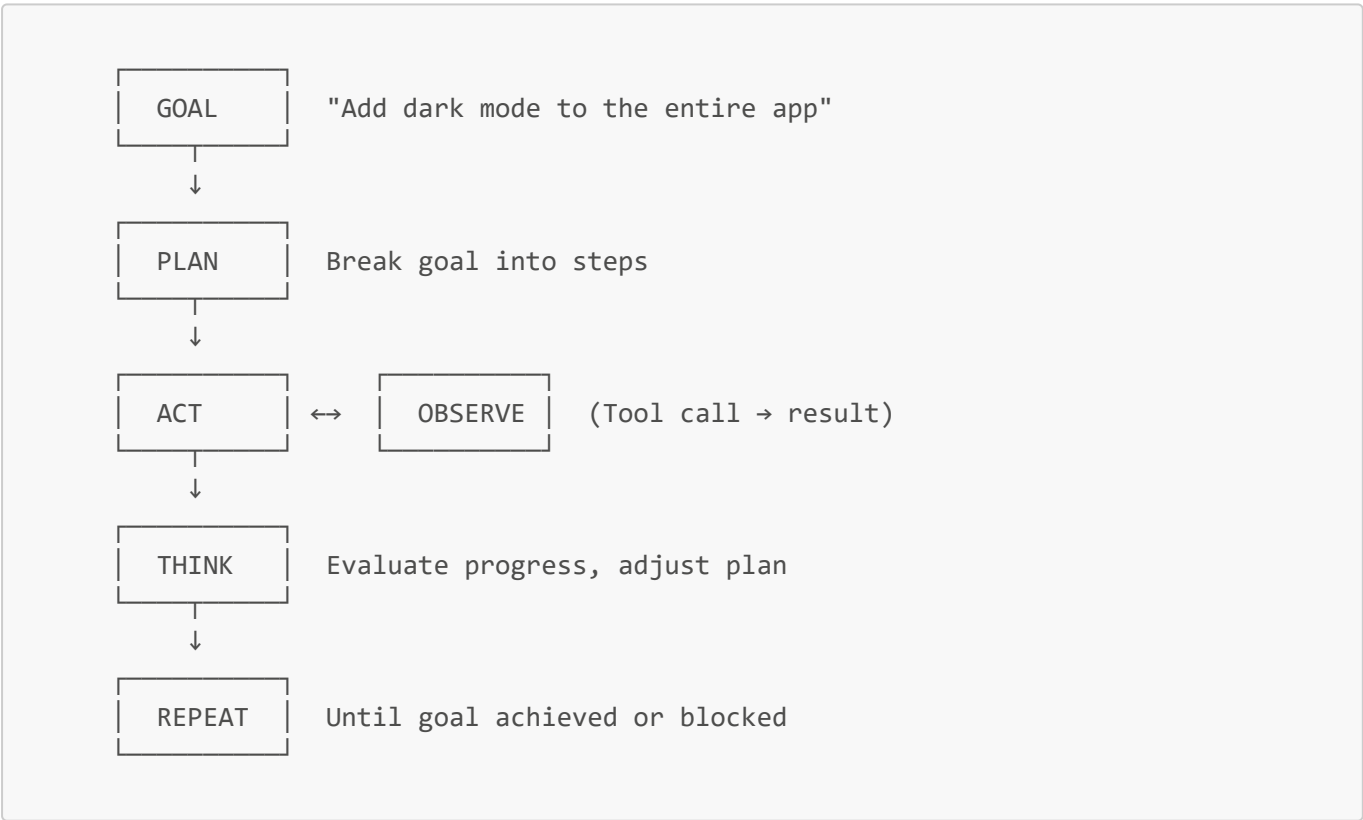
Agents

Concept · 8 minutes

Agent vs. Chatbot

Aspect	Chatbot	Agent
Interaction	Single turn or simple back-and-forth	Multi-step, goal-oriented
Control	User drives each step	Agent plans and executes
Memory	Limited to conversation	Can maintain state across steps
Actions	None (text only)	Can call tools, modify files
Autonomy	None	Goal-directed autonomy
Example	"Explain this code"	"Fix all bugs in this repository"

The Agent Loop



Levels of Agent Autonomy

Level	Name	Description	Example
L1	Assistant	Responds, needs step-by-step guidance	Basic chatbot
L2	Tool-caller	Can request tools, human approves	Cursor Agent with approval
L3	Planner	Makes plans, executes with supervision	Auto-code review
L4	Autonomous	Self-directed, minimal supervision	CI/CD agent
L5	Full Agent	Complete task ownership	Enterprise automation

How Agents Change Your Role

Traditional:

Developer writes every line → tests → deploys

Agent-Assisted:

Developer defines intent → Agent executes → Developer reviews → Agent iterates

Old Role	New Role
Code writer	Intent specifier
Debugger	Quality reviewer
Implementation	Orchestration
Manual testing	Acceptance testing
Problem solver	Problem framer

When to Use Agents

Good for agents:

- Large, multi-step tasks · Repetitive patterns
- Well-defined with clear success criteria
- Low-risk changes · Documentation updates

Bad for agents:

- Security-critical systems · Unrecoverable actions
- Poorly defined goals · Real-time requirements
- High cost of failure

Module Summary

Lesson	Topic	Key Insight
1.1	How AI Models Work	Probabilistic, not deterministic – manage with temperature
1.2	Hallucinations	Models invent confidently – always verify
1.3	Tokens and Pricing	Output costs more – optimize context, use cheaper models
1.4	Context	Single most valuable skill – quality in = quality out
1.5	Tool Calling & MCP	AI requests actions, you control execution
1.6	Agents	Goal-directed action – changes developer role

Cursor Editor Essentials

Module 2 · Day 1 (Hands-On)

Cursor Training Program · Hands-on exercise · ~90 min

Module Overview

Aspect	Details
Duration	~90 minutes
Format	Hands-on exercise
Prerequisites	Module 1 completed, Cursor installed, Git repository access
Module Goal	Master the core workflows of AI-assisted coding in Cursor

Learning Objectives

By the end of this module, participants will be able to:

- Orient an AI agent to an unfamiliar codebase
- Get targeted explanations of specific files or symbols
- Make safe, reviewable changes using diff review
- Design complex changes with Plan Mode
- Compare models to choose the right one for each task
- Use @mentions for precise context control
- Navigate checkpoints as a safety net
- Let agents run terminal commands and react to output

Agenda

Lesson	Topic	Time
2.1	Codebase Understanding	20 min
2.2	Explaining Files/Symbols	13 min
2.3	Safe Reviewable Changes	13 min
2.4	Plan Mode	13 min
2.5	Comparing Models	13 min
2.6	@mentions	13 min
2.7	Checkpoints	8 min
2.8	Terminal Integration	13 min

Lesson 2.1

Codebase Understanding

Concept · 8 min · Exercise · 12 min

The Problem & The Solution

The Problem: Opening a new codebase is overwhelming. Where do you start? What's the entry point?

The Cursor Solution: Ask the agent to explain the codebase. It reads files, traces connections, and returns a roadmap.

"Drop an agent into a codebase you've never seen and get a coherent explanation of how it works."

Windows Exercise Environment

All exercises in this module assume **Windows 10/11** with Cursor installed.

Terminal	Use when	Open in Cursor
PowerShell	Default — Python, Git, <code>curl.exe</code> , npm, Cursor CLI (<code>agent</code>)	<code>Ctrl+`</code> → PowerShell
Git Bash	Bash syntax, <code>export VAR=...</code> , shell scripts ending in <code>.sh</code>	Terminal menu → Git Bash
Command Prompt	Legacy <code>.bat</code> files only	Terminal menu → Command Prompt
Ubuntu (WSL)	Linux-only tools or native bash without Git Bash	Terminal menu → Ubuntu (WSL)

Cursor Agent panel (**Ctrl+L**) is for natural-language prompts — not a shell.

Set default profile: Settings → `terminal.integrated.defaultProfile.windows` → **PowerShell**

Exercise 2.1 — Steps 1–2

Platform: Windows 10/11 · Prompts → **Agent panel** **Ctrl+L** · Agent commands → **PowerShell** **Ctrl+`**

Step 1: Open an unfamiliar repository in Cursor — **Terminal: PowerShell** (**Ctrl+`**)

1. Open terminal in Cursor (**Terminal** → **New Terminal** → profile **PowerShell**)
2. Clone the sample repo and open it in Cursor:

```
git clone https://github.com/facebookresearch/detectron2
cd detectron2
cursor .
```

Exercise 2.1 — Steps 1–2 (Part 2)

Step 2: Open the Agent — **Where: Cursor Agent panel** — **Ctrl+L** (or **Ctrl+I** for inline Agent in editor)

Exercise 2.1 — Step 3: Orientation Prompt

Platform: Windows 10/11 · Prompts → **Agent panel** **Ctrl+L** · Diffs → **Editor**

```
Explain this codebase to me as if I'm a new team member.

Specifically tell me:
1. What is the main purpose of this project?
2. What are the entry points (main scripts, CLI, API)?
3. What are the key modules and how do they relate?
4. What are the main dependencies?
5. What files should I read first to understand the architecture?
```

Exercise 2.1 — Steps 4–5

Platform: Windows 10/11 · Prompts → **Agent panel** **Ctrl+L** · Diffs → **Editor**

Step 4: Follow up — trace data flow: **Where: Cursor Agent panel** — **Ctrl+L**

```
Based on what you just told me, trace the flow of data from input
to output. What functions get called in order?
```

Exercise 2.1 — Steps 4–5 (Part 2)

Step 5: Ask for a visual overview: **Where: Cursor Agent panel** — **Ctrl+L** (or **Ctrl+I** for inline Agent)

Create an ASCII diagram showing the module relationships in this codebase.

Expected Agent Output (Sample)

PURPOSE: Object detection and segmentation library (PyTorch)

ENTRY POINTS:

- train_net.py – Main training script
- demo.py – Inference demo

KEY MODULES:

config/ ↔ model_zoo ↔ engine/

↓

↓

↓

structures ↔ modeling/ ↔ data/

DATA FLOW:

Config → build_model() → DataLoader → Training Loop → Loss → Backprop

Pro Tip & Success Criteria

Pro Tip: Save the agent's explanation as a project note:

Save this explanation as `.cursor/project-overview.md` so future team members can read it.

Success Criteria:

- Agent described project purpose
- Agent identified entry points and key modules
- Agent suggested first files to read

Lesson 2.2

Explaining a Specific File or Symbol

Concept · 5 min · Exercise · 8 min

Targeted Explanations

"Don't make the agent read the whole codebase when you just need to understand one function."

Use **precise context** — select a function or class, then ask focused questions.

Exercise 2.2 — Steps 1–3

Platform: Windows 10/11 · Prompts → **Agent panel** **Ctrl+L** · Diffs → **Editor**

Step 1: Open a specific file in your project **Where:** **Cursor Agent panel** — **Ctrl+L** (or **Ctrl+I** for inline Agent)

Step 2: Select a function or class you want explained **Where:** **Cursor Agent panel** — **Ctrl+L** (or **Ctrl+I** for inline Agent)

Exercise 2.2 — Steps 1–3 (Part 2)

Step 3: Use the Agent with precise context: **Where:** **Cursor Agent panel** — **Ctrl+L**

```
Explain the function I have selected. For each major section, tell me:  
- What it does  
- Why it's designed that way (trade-offs)  
- Potential edge cases or bugs  
- How it could be improved
```

Exercise 2.2 — Steps 4–5

Platform: Windows 10/11 · Prompts → **Agent panel** **Ctrl+L** · Diffs → **Editor**

Step 4: Ask for a concrete example: **Where:** **Cursor Agent panel** — **Ctrl+L** (or **Ctrl+I** for inline Agent)

```
Give me a concrete example of inputs and outputs for this function.  
Show me what happens in the normal case and one edge case.
```

Exercise 2.2 — Steps 4–5 (Part 2)

Step 5: Ask about dependencies: **Where:** **Cursor Agent panel** — **Ctrl+L** (or **Ctrl+I** for inline Agent)

```
What other functions does this call? What calls this function?  
Trace the call chain two levels in each direction.
```

Inline Explanation Shortcut

```
# Select code, press Cmd+L (or Ctrl+L)
# The agent explains the selected code in the chat panel
```

Success Criteria:

- Selected specific code · Agent explained the selection
- Agent provided input/output examples · Agent traced call dependencies

Lesson 2.3

Making a Safe, Reviewable Change

Concept · 5 min · Exercise · 8 min

The Diff Review Workflow

1. Ask agent to propose a change
2. Review the diff (what's added/removed)
3. Accept or reject changes
4. Test after acceptance

"Before AI changes your code, see exactly what will change and approve it."

Exercise 2.3 — Steps 1–2

Platform: Windows 10/11 · Prompts → **Agent panel** **Ctrl+L** · Diffs → **Editor**

Step 1: Ask for a small, safe change: **Where: Cursor Agent panel** — **Ctrl+L** (or **Ctrl+I** for inline Agent)

```
Change the welcome message in index.html from "Hello World"
to "Welcome to My App"
```

Exercise 2.3 — Steps 1–2 (Part 2)

Step 2: Watch the agent generate the diff: **Where: Cursor Agent panel** — **Ctrl+L** (or **Ctrl+I** for inline Agent)

 Changes to index.html:

```
<h1>- Hello World</h1>
```

```
<h1>+ Welcome to My App</h1>
```

Accept? [Yes] [No] [Edit]

Exercise 2.3 — Review Questions

Platform: Windows 10/11 · **PowerShell** `Ctrl+`` (Git Bash/WSL for `.sh` scripts)

Before accepting, ask yourself:

- Are the changes **only** what I asked for?
- Are there unexpected additions or deletions?
- Does the syntax look correct?
- Will this break anything else?

Step 4: Accept · **Step 5:** Test manually **Where:** Cursor Agent panel — `Ctrl+L`

```
open index.html      # web
python script.py     # Python
npm start            # React
```

Exercise 2.3 — If Something Goes Wrong

Platform: Windows 10/11 · Prompts → **Agent panel** `Ctrl+L` · Diffs → **Editor**

```
That change didn't work. The button disappeared.
Please explain what happened and suggest a fix.
```

Success Criteria:

- Agent proposed a change · Reviewed diff before accepting
- Accepted only after verification · Tested the change

Lesson 2.4

Plan Mode

Concept · 5 min · Exercise · 8 min

Design Before You Code

Plan Mode makes the agent create a **detailed plan BEFORE writing any code.**

When to use Plan Mode:

- Changing multiple files · Adding a new feature
- Refactoring existing code
- You're not 100% sure of the best approach
- The change is risky or hard to undo

Exercise 2.4 — Enable Plan Mode

Platform: Windows 10/11 · **PowerShell** `Ctrl+`` (Git Bash/WSL for `.sh` scripts)

```
# Press Shift+Tab in the Agent input
# The input border changes color to indicate Plan Mode
```

Step 2: Describe a complex change: **Terminal: PowerShell** — unless step notes Git Bash or WSL

Add user authentication to this web app.

Requirements:

- Email/password login · Session management
- Protected routes (dashboard, settings)
- Logout functionality · "Remember me" option

Don't write code yet – just give me a plan.

What a Good Plan Looks Like

IMPLEMENTATION PLAN

Step 1: Create User Model – `models/user.js`
Step 2: Auth Routes – `routes/auth.js` (login, logout, register)
Step 3: Session Management – `middleware/session.js`
Step 4: Protected Route Middleware – `middleware/auth.js`
Step 5: Update Frontend – `pages/login.html`, `dashboard.html`
Step 6: Environment Variables – `.env` (`SESSION_SECRET`, `REDIS_URL`)

Questions for you:

1. JWT or server-side sessions?
2. Existing user database?
3. Include email verification?

Ready to proceed? [Yes] [No] [Modify Plan]

Exercise 2.4 — Approve & Execute

Platform: Windows 10/11 · Prompts → **Agent panel** `Ctrl+L` · Diffs → **Editor**

Step 4: Answer questions and approve: **Where:** **Cursor Agent panel** — `Ctrl+L` (or `Ctrl+I` for inline Agent)

```
Use JWT for simplicity. No existing database yet - use SQLite for now.
Skip email verification for this version. Proceed.
```

Exercise 2.4 — Approve & Execute (Part 2)

Step 5: Watch the agent execute the plan step by step **Where:** **Cursor Agent panel** — `Ctrl+L`

Success Criteria:

- Enabled Plan Mode (Shift+Tab) · Agent created structured plan
- Agent asked clarifying questions · Approved plan before code was written

Lesson 2.5

Comparing Two Models

Concept · 5 min · Exercise · 8 min

Model Selection Guide

Task Type	Recommended Model	Why
Typo fixes, simple edits	GPT-5 Mini	Cheap, fast, good enough
Daily coding, bug fixes	GPT-5.3 Codex or Claude Sonnet	Best value, high quality
Complex logic, architecture	Claude Opus or GPT-5.5	Smartest, but expensive
Frontend/visual work	Gemini 3.1 Pro	Can see images
Fast, simple questions	Claude Haiku	Fastest responses

Exercise 2.5 — Compare Two Models

Platform: Windows 10/11 · Prompts → **Agent panel** `Ctrl+L` · Diffs → **Editor**

Step 1: Set model to Claude Sonnet, ask: **Where:** **Cursor Agent panel** — `Ctrl+L`

```
Explain what a closure is in JavaScript with a practical example.
```

Exercise 2.5 — Compare Two Models (Part 2)

- Step 2:** Copy the response **Where: Cursor Agent panel** — `Ctrl+L`
- Step 3:** Switch to GPT-5 Mini — ask the **same question Where: Cursor Agent panel** — `Ctrl+L` (or `Ctrl+I` for inline Agent)

Exercise 2.5 — Compare Two Models (Part 3)

- Step 4:** Compare responses **Where: Cursor Agent panel** — `Ctrl+L` (or `Ctrl+I` for inline Agent)

Comparison Point	Claude Sonnet	GPT-5 Mini
Length		
Code example quality		
Explanation clarity		
Speed		

Exercise 2.5 — Cost & Decision Matrix

- Platform:** Windows 10/11 · Prompts → **Agent panel** `Ctrl+L` · Diffs → **Editor**
- Step 5:** Check token usage at bottom of chat after each request **Where: Cursor Agent panel** — `Ctrl+L`

Exercise 2.5 — Cost & Decision Matrix (Part 2)

- Step 6:** Create a personal decision matrix: **Where: Cursor Agent panel** — `Ctrl+L`

If task is...	Use model...
Typos / text changes	→ GPT-5 Mini
Quick question	→ Claude Haiku
Daily coding	→ GPT-5.3 Codex
Complex debugging	→ Claude Sonnet
Architecture design	→ Claude Opus
UI/frontend from image	→ Gemini Pro

Lesson 2.6

Precise Context with @mentions

Concept · 5 min · Exercise · 8 min

@mention Types

@mention	What It Does	Example
@filename	Include specific file	@auth.py
@symbol	Include function/class	@UserModel
@branch	Reference git branch	@main
@chat	Reference past conversation	@previous-chat
@folder	Reference entire directory	@/src/utils
@web	Search the web	@web pandas DataFrame

"Laser-targeting instead of spraying the whole codebase."

Exercise 2.6 — @filename & @symbol

Platform: Windows 10/11 · Prompts → **Agent panel** `Ctrl+L` · Diffs → **Editor**

Step 1: Target a specific file: **Where:** **Cursor Agent panel** — `Ctrl+L`

```
@database.py What are the security vulnerabilities in this database connection?
```

Exercise 2.6 — @filename & @symbol (Part 2)

Step 2: Target a specific function: **Where:** **Cursor Agent panel** — `Ctrl+L`

```
@calculate_total This function is returning NaN sometimes. Why?
```

Exercise 2.6 — @filename & @symbol (Part 3)

Step 3: Combine multiple @mentions: **Where:** **Cursor Agent panel** — `Ctrl+L` (or `Ctrl+I` for inline Agent)

```
@auth.py @UserModel @login_handler Review the authentication flow.  
Are there any race conditions or timing attacks?
```

Exercise 2.6 — @branch, @chat, @folder, @web

Platform: Windows 10/11 · Prompts → **Agent panel** `Ctrl+L` · Diffs → **Editor**

Compare @main and @feature/payment branches.
What are the key differences in the payment handling code?

@chat(authentication-discussion) Based on that discussion,
implement the fix we agreed on.

@src/components Find all components that don't have loading states.

@web React 19 useTransition hook How do I use it?

@mention Pro Tips

- Start typing @ — Cursor auto-suggests available mentions
- You can @mention **multiple items** in one message
- @mentions work in both **Agent** and **Chat** modes

Success Criteria:

- Used @filename, @symbol, multiple @mentions, and @web

Lesson 2.7

Checkpoints

Concept · 4 min · Exercise · 4 min

A Safety Net for Experiments

What Checkpoints Save:

- Code changes made by the agent
- Conversation history · File states

When to Create Checkpoints:

- Before complex changes · At milestones (Step 2 of 5)
- Before risky experiments · Before terminal commands

Exercise 2.7 — Create & Restore

Platform: Windows 10/11 · **PowerShell** `Ctrl+`` (Git Bash/WSL for `.sh` scripts)

Step 1: Create a checkpoint before making a change **Where:** **Cursor Agent panel** — `Ctrl+L` (or `Ctrl+I` for inline Agent)

```
# Click checkpoint icon in Agent panel
# Or: Cmd+Shift+S / Ctrl+Shift+S
```

Exercise 2.7 — Create & Restore (Part 2)

Step 2: Name it descriptively: `"Before auth refactor - safe point"` **Terminal:** **PowerShell** — unless step notes Git Bash or WSL

Exercise 2.7 — Create & Restore (Part 3)

Step 3: Let the agent make changes: **Terminal:** **PowerShell** — `Ctrl+`` in Cursor

```
Add input validation to all form handlers.
```

Exercise 2.7 — Create & Restore (Part 4)

Step 4: If something goes wrong → **Restore to checkpoint** **Where:** **Cursor Agent panel** — `Ctrl+L` (or `Ctrl+I` for inline Agent)

Step 5: View history via the clock icon in Agent panel **Where:** **Cursor Agent panel** — `Ctrl+L` (or `Ctrl+I` for inline Agent)

Checkpoint Best Practices

- Create checkpoints every 5–10 minutes during complex work
- Use **descriptive names**, not "checkpoint1"
- Test the restored state before continuing
- Clean up old checkpoints periodically

Success Criteria:

- Created checkpoint · Made changes · Restored · Verified restoration

Lesson 2.8

Terminal Integration

Concept · 5 min · Exercise · 8 min

What the Agent Can Do

- Run shell commands · See stdout, stderr, exit codes
- React to command output · Install dependencies
- Run tests · Start/stop services

Safety Features:

- You approve each command before execution
 - Commands appear in terminal for you to see
 - You can reject dangerous commands
-

Exercise 2.8 — Steps 1–3

Platform: Windows 10/11 · Prompts → **Agent panel** `Ctrl+L` · Diffs → **Editor**

Step 1: Check the environment: **Where:** **Cursor Agent panel** — `Ctrl+L`

```
Run `python --version` and tell me what Python version we're using.
```

Exercise 2.8 — Steps 1–3 (Part 2)

Step 2: Approve the command when prompted **Where:** **Cursor Agent panel** — `Ctrl+L` (or `Ctrl+I` for inline Agent)

Exercise 2.8 — Steps 1–3 (Part 3)

Step 3: Run tests and fix failures: **Where:** **Cursor Agent panel** — `Ctrl+L`

```
Run the test suite. If any tests fail, fix them.  
Show me what you're changing.
```

Exercise 2.8 — Agent Terminal Loop

Platform: Windows 10/11 · Prompts → **Agent panel** `Ctrl+L` · Agent commands → **PowerShell** `Ctrl+``

```
Agent: I'll run the tests first.  
→ Executing: pytest tests/  
→ Output: 7 passed, 2 failed
```

```
Agent: Two tests failed. Let me look at them.  
→ Reading: tests/test_auth.py  
  
Agent: Login test failing – timeout too short. Increasing 5 → 10 seconds.  
→ Editing: tests/test_auth.py  
  
→ Executing: pytest tests/test_auth.py  
→ Output: All tests passed
```

Exercise 2.8 — More Commands

Platform: Windows 10/11 · Prompts → **Agent panel** `Ctrl+L` · Diffs → **Editor**

Step 5: Install a dependency: **Where:** **Cursor Agent panel** — `Ctrl+L`

```
Install the 'requests' library if it's not already installed.
```

Exercise 2.8 — More Commands (Part 2)

Step 6: Multi-step workflow: **Where:** **Cursor Agent panel** — `Ctrl+L`

```
Run these commands in order:  
1. git checkout main 2. git pull  
3. git checkout -b feature/analytics  
4. Create analytics.py 5. Run flake8 6. Fix style issues  
  
Confirm before each command that might affect the repo.
```

Terminal Command Safety Rules

Category	Commands
Always approve first	<code>rm</code> , <code>sudo</code> , <code>git push --force</code> , production changes
Review carefully	<code>npm install</code> , <code>pip install</code> , git branch changes, docker
Safe to auto-approve	<code>python --version</code> , <code>ls</code> , <code>pwd</code> , <code>cat</code> , <code>pytest</code> , <code>npm test</code>

Success Criteria:

- Ran version check · Ran tests and reacted to output
- Installed dependency · Executed multi-step workflow

Module Summary

Lesson	Topic	Key Skill
2.1	Codebase Understanding	Orient to new repo
2.2	Explaining Files/Symbols	Targeted explanations
2.3	Safe Reviewable Changes	Diff review workflow
2.4	Plan Mode	Design before code
2.5	Comparing Models	Model selection
2.6	@mentions	Precise context
2.7	Checkpoints	Safety net
2.8	Terminal Integration	Command execution

Quick Reference Card

SHORTCUTS:

Cmd/Ctrl + I

→ Open Agent

Cmd/Ctrl + L

→ Explain selected code

Shift + Tab

→ Toggle Plan Mode

Cmd/Ctrl + Shift + S

→ Create checkpoint

@MENTIONS: @filename @symbol @branch @chat @web

SAFE CHANGE: Ask → Review diff → Accept/reject → Test

PLAN MODE: Shift+Tab → Describe → Review plan → Approve → Execute

Agent Modes and Tools

Module 3 · Day 1 (Hands-On + Concept)

Cursor Training Program · ~60 min

Module Overview

Aspect	Details
Duration	~60 minutes
Format	Hands-on exercise + concept
Prerequisites	Module 2 completed, live web app available (or sample provided)
Module Goal	Master different agent modes and the core tools that make agents powerful

Learning Objectives

By the end of this module, participants will be able to:

- Choose between Ask Mode and Agent Mode based on task and safety needs
- Use the Browser Tool to inspect live pages and read console output
- Run terminal commands through the agent and diagnose failures
- Write effective, constrained prompts that avoid scope creep

Agenda

Lesson	Topic	Time
3.1	Ask Mode vs. Agent Mode	18 min
3.2	Browser Tool	18 min
3.3	Terminal Tool	20 min
3.4	Effective Prompting in Practice	22 min

Lesson 3.1

Ask Mode vs. Agent Mode

Concept · 10 min · Exercise · 8 min

The Core Distinction

Aspect	Ask Mode	Agent Mode
Can read files	✔ Yes (with @mentions)	✔ Yes
Can edit files	✘ No	✔ Yes
Can run terminal	✘ No	✔ Yes
Can browse web	✘ No (limited)	✔ Yes (with tool)
Can call tools	✘ No	✔ Yes
Safety level	Very high (read-only)	Moderate (needs oversight)
Best for	Questions, learning, code review	Implementation, debugging, automation

When to Use Each Mode

USE ASK MODE when:

- You have a question about code · Exploring a codebase

- You want a second opinion on design
- You're not ready to make changes · Production environment

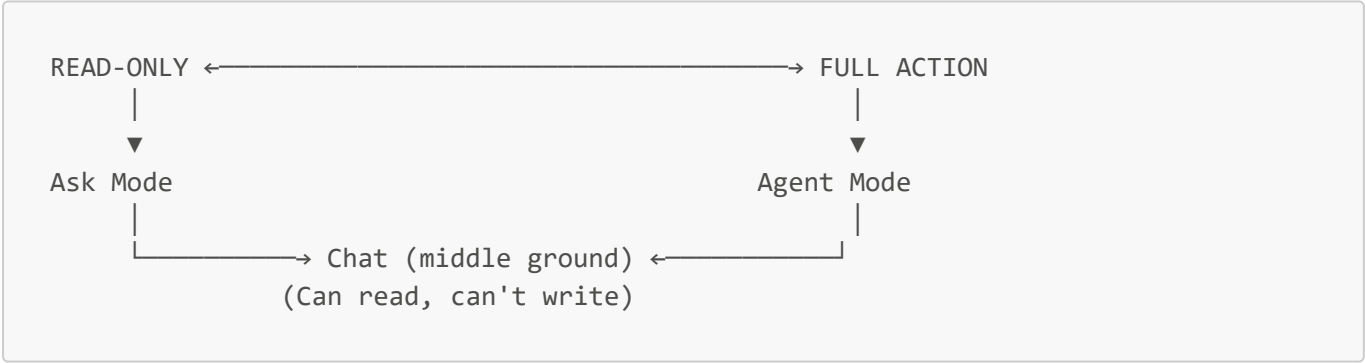
USE AGENT MODE when:

- You want the AI to write/change code
- You need to run and react to commands
- Multi-step tasks · Development environment
- You're prepared to review changes

Safety Implications

Risk	Ask Mode	Agent Mode
Unintended code changes	None	Moderate (requires review)
File deletion	None	Possible (needs approval)
Malicious commands	None	Possible (needs approval)
Data leakage	Low	Medium (can read files)
API cost	Low (no tool calls)	Higher (multiple tool calls)

The Mode Continuum



"Not every AI interaction needs full agent capabilities."

Windows Exercise Environment

All exercises in this module assume **Windows 10/11** with Cursor installed.

Terminal	Use when	Open in Cursor
PowerShell	Default — Python, Git, <code>curl.exe</code> , npm, Cursor CLI (<code>agent</code>)	<code>Ctrl+`</code> → PowerShell
Git Bash	Bash syntax, <code>export VAR=...</code> , shell scripts ending in <code>.sh</code>	Terminal menu → Git Bash

Terminal	Use when	Open in Cursor
Command Prompt	Legacy <code>.bat</code> files only	Terminal menu → Command Prompt
Ubuntu (WSL)	Linux-only tools or native bash without Git Bash	Terminal menu → Ubuntu (WSL)

Cursor Agent panel (`Ctrl+L`) is for natural-language prompts — not a shell.

Set default profile: Settings → `terminal.integrated.defaultProfile.windows` → **PowerShell**

Exercise 3.1 — Steps 1–2

Platform: Windows 10/11 · Prompts → **Agent panel** `Ctrl+L` · Diffs → **Editor**

Step 1: Open Agent panel (`Cmd+I` / `Ctrl+I`) — note mode indicator at bottom **Where: Cursor Agent panel** — `Ctrl+L` (or `Ctrl+I` for inline Agent)

Exercise 3.1 — Steps 1–2 (Part 2)

Step 2: Try to make a change in **Ask Mode: Where: Cursor Agent panel** — `Ctrl+L` (or `Ctrl+I` for inline Agent)

```
Change the variable name 'temp' to 'temperature' in the current file.
```

Exercise 3.1 — Steps 3–5

Platform: Windows 10/11 · Prompts → **Agent panel** `Ctrl+L` · Diffs → **Editor**

Step 3: Ask a question Ask Mode handles well: **Where: Cursor Agent panel** — `Ctrl+L` (or `Ctrl+I` for inline Agent)

```
Explain the purpose of the main() function in this file.
What edge cases does it handle?
```

Exercise 3.1 — Steps 3–5 (Part 2)

Step 4: Switch to **Agent Mode** via the dropdown **Where: Cursor Agent panel** — `Ctrl+L`

Step 5: Repeat the rename request — agent shows diff for approval **Where: Cursor Agent panel** — `Ctrl+L` (or `Ctrl+I` for inline Agent)

Exercise 3.1 — Step 6 & Success Criteria

Platform: Windows 10/11 · Prompts → **Agent panel** `Ctrl+L` · Diffs → **Editor**

Step 6: Practice mode-switching mid-conversation: **Where:** **Cursor Agent panel** — `Ctrl+L`

```
# Start in Ask Mode: What does this function return?
# Then: "Switch to Agent Mode and fix the off-by-one error"
```

Success Criteria:

- Used Ask Mode for questions · Observed Ask Mode cannot make changes
- Switched to Agent Mode · Made a change with diff review

Lesson 3.2

Browser Tool

Concept · 8 min · Exercise · 10 min

What the Browser Tool Can Do

- Navigate to URLs · Read page content and DOM structure
- See console logs and errors · Take screenshots (depending on model)
- Click elements and interact with pages
- Extract data from live pages

"See what your app actually looks like in a browser — not just the source code."

Browser Tool: With vs. Without

Scenario	Without Browser	With Browser
"Why is the button not showing?"	Guesses from CSS	Sees the rendered page
"Is the API returning data?"	Checks code	Sees network tab
"What console errors?"	Asks you	Reads console directly
"Does responsive layout work?"	Trusts CSS	Views at different sizes

Exercise 3.2 — Steps 1–2

Platform: Windows 10/11 · **PowerShell** `Ctrl+`` (Git Bash/WSL for `.sh` scripts)

Step 1: Start a local web app (or use a public test page) **Terminal: PowerShell** — unless step notes Git Bash or WSL

```
python -m http.server 8000
# Or use a public test page
```

Exercise 3.2 — Steps 1–2 (Part 2)

Step 2: In Agent Mode: **Terminal: PowerShell** — **Ctrl+`** in Cursor

```
Use the browser tool to open http://localhost:8000
Tell me what you see on the page.
```

Exercise 3.2 — Steps 3–4

Platform: Windows 10/11 · Agent → **Ctrl+L** · Shell → **PowerShell** · Browser for dashboards

Step 3: Find specific elements: **Where: Cursor Agent panel** — **Ctrl+L**

```
On that same page, find:
1. The main heading text
2. The number of buttons
3. Any error messages visible
```

Exercise 3.2 — Steps 3–4 (Part 2)

Step 4: Check the console: **Where: Cursor Agent panel** — **Ctrl+L**

```
Now open the browser developer console.
Are there any errors or warnings? If so, what are they?
```

Expected Agent Actions

```
→ browser_navigate(url="http://localhost:8000")
→ browser_snapshot() – page structure captured
→ browser_console_messages()
  [WARNING] Deprecated API used on line 42
  [ERROR] Failed to load resource: /api/data 404
```

Agent: Found a deprecated API warning and a 404 on /api/data

Exercise 3.2 — Steps 5–6

Platform: Windows 10/11 · Agent → **Ctrl+L** · Shell → **PowerShell** · Browser for dashboards

Step 5: Diagnose a layout issue: **Where: Cursor Agent panel** — **Ctrl+L**

The login button is partially hidden on mobile sizes.
Use the browser tool to check what's happening.

Exercise 3.2 — Steps 5–6 (Part 2)

Step 6: Extract data from a page: **Where: Cursor Agent panel** — **Ctrl+L**

Go to `https://example.com/pricing`
Extract all pricing plan names and their monthly costs into a table.

Browser Tool Limitations

Limitation	Workaround
Cannot log in to sites	Provide login instructions or session cookies
JavaScript-heavy sites may load slowly	Add wait instructions
Rate limits on some sites	Space out requests
Cannot upload files	Not supported yet

Success Criteria: Opened URL · Read content · Checked console · Extracted data

Lesson 3.3

Terminal Tool

Concept · 8 min · Exercise · 12 min

What the Terminal Tool Can Do

- Run any shell command (with approval)

- See stdout, stderr, exit codes
- Read command output as context for next actions
- Chain commands based on previous results

Terminal Tool Flow

```
User: "Run the tests and fix any failures"
→ Agent: Execute "pytest tests/"
→ Result: Exit code 1, "2 failed, 5 passed"
→ Agent: Reads failures, fixes code
→ Agent: Reruns tests
→ Agent: "All tests passed."
```

Exercise 3.3 — Steps 1–2

Platform: Windows 10/11 · Prompts → **Agent panel** `Ctrl+L` · Diffs → **Editor**

Step 1: Check your environment: **Where:** **Cursor Agent panel** — `Ctrl+L`

```
Run these commands and tell me what versions we're using:
- python --version
- node --version (if applicable)
- git --version
```

Exercise 3.3 — Steps 1–2 (Part 2)

Step 2: Run the test suite: **Where:** **Cursor Agent panel** — `Ctrl+L`

```
Run the test suite. Show me which tests pass and which fail.
```

Exercise 3.3 — Steps 3–4

Platform: Windows 10/11 · Prompts → **Agent panel** `Ctrl+L` · Diffs → **Editor**

Step 3: Diagnose failures: **Where:** **Cursor Agent panel** — `Ctrl+L`

```
One or more tests failed. What's causing these failures?
Look at the specific error messages.
```

Exercise 3.3 — Steps 3–4 (Part 2)

Step 4: Fix and verify: **Where:** **Cursor Agent panel** — **Ctrl+L**

Based on your diagnosis, fix the failing tests.
Show me what you're changing before you run again.

Exercise 3.3 — Debug Workflow (Step 5)

Platform: Windows 10/11 · Prompts → **Agent panel** **Ctrl+L** · Diffs → **Editor**

I found a bug where the app crashes when input is empty.

1. First, run the app to reproduce the crash
2. Then, add logging to understand why
3. Finally, fix the bug and verify it works

→ python app.py → IndexError: list index out of range (line 23)
→ Agent adds guard condition
→ python app.py --test-empty-input → "No data provided"

Exercise 3.3 — Step 6 & Safety Rules

Platform: Windows 10/11 · Prompts → **Agent panel** **Ctrl+L** · Diffs → **Editor**

Step 6: React to long-running commands: **Where:** **Cursor Agent panel** — **Ctrl+L**

Run npm install or pip install. Watch the output.
If there's a warning about deprecated packages, note it and suggest fixes.

Approval Required	No Approval Needed
Writes files, <code>sudo</code> , <code>git push --force</code>	Version checks, <code>cat</code> , <code>ls</code>
<code>npm install -g</code> , start servers	<code>pytest</code> , <code>npm test</code> , <code>git status</code>

Success Criteria: Ran tests · Diagnosed failure · Fixed code · Verified fix

Lesson 3.4

Effective Prompting in Practice

Concept · 10 min · Exercise · 12 min

Anatomy of an Effective Prompt

- 1. **ROLE / CONTEXT** — "You are a senior Python developer..."
- 2. **TASK** — "Fix the bug in calculate_total()..."
- 3. **CONSTRAINTS** — "Do not change the function signature..."
- 4. **OUTPUT FORMAT** — "Show me the diff and explain your change..."
- 5. **SUCCESS CRITERIA** — "Function should return 0 for empty input..."

Bad Prompts vs. Good Prompts

Bad Prompt	Good Prompt
"Fix this code"	"Fix the IndexError in process_list() when list is empty. Do not change return type."
"Add logging"	"Add INFO-level logging to calculate() using existing logger config."
"Make it faster"	"Optimize find_user() from O(n²) to O(n log n). Don't change signature."
"Review my code"	"Review auth.py for SQL injection, password handling, session issues. Ignore style."

The "Boundaries" Technique

Always tell the agent what **NOT** to touch:

BOUNDARIES:

- Do NOT change: function signatures, return types, existing tests
- Do NOT touch: config files, database schemas, other modules
- Do NOT delete: comments, logging, error handling
- Do NOT add: new dependencies, external APIs, global state

Change ONLY: the function body of calculate_total()

Avoiding Scope Creep

The problem:

User: "Fix the login bug."

Agent: "Fixed login. Also refactored auth, added OAuth, reorganized codebase."

User: "Wait, I just wanted the login bug fixed!"

Technique	Example
Explicit boundaries	"Change ONLY login.js lines 42–56"
One thing at a time	"First, just identify the issue. Don't fix yet."
Ask for plan first	"Plan Mode: Show me what you'll change before doing it"
Use checkpoints	Create checkpoint before complex requests
Prefer diffs	"Show me the diff, don't replace the whole file"

Exercise 3.4 — Step 1: Constrained Prompt

Platform: Windows 10/11 · Prompts → **Agent panel** `Ctrl+L` · Diffs → **Editor**

Task: Fix the bug where `get_user_email(user_id)` returns `None` for valid users.

Constraints:

- Do NOT change the function signature
- Do NOT add new imports
- Do NOT modify other functions
- Do NOT add print statements (use existing logger)

Output format: Show exact diff and explain root cause.

Success criteria: Function returns email string for valid user IDs.

Exercise 3.4 — Steps 2–3

Platform: Windows 10/11 · Prompts → **Agent panel** `Ctrl+L` · Diffs → **Editor**

Step 2: Compare constrained vs. unconstrained: **Where:** **Cursor Agent panel** — `Ctrl+L` (or `Ctrl+I` for inline Agent)

Fix `get_user_email` - it's returning `None` sometimes.

Exercise 3.4 — Steps 2–3 (Part 2)

Step 3: Plan first: **Where:** **Cursor Agent panel** — `Ctrl+L`

Before making any changes, answer:

1. What files would need to change?
2. What is the root cause you suspect?
3. What are the risks?
4. Are there alternative approaches?

I will review before approving any code changes.

Exercise 3.4 — Steps 4–5

Platform: Windows 10/11 · Prompts → **Agent panel** `Ctrl+L` · Diffs → **Editor**

Step 4: Negative constraints: **Where:** **Cursor Agent panel** — `Ctrl+L`

Add error handling to the file parser.

But DO NOT:

- Change the return type (must remain dict)
- Add external dependencies
- Swallow exceptions silently (always log them)
- Change the existing test file

Exercise 3.4 — Steps 4–5 (Part 2)

Step 5: One change at a time: **Where:** **Cursor Agent panel** — `Ctrl+L`

First, add input validation. Just show me what you'd add – don't modify yet.
[Review] Now add the validation. Show the diff before I accept.

Exercise 3.4 — Step 6: Prompt Templates

Platform: Windows 10/11 · Prompts → **Agent panel** `Ctrl+L` · Diffs → **Editor**

Save as `.cursor/prompt-templates.md`:

```
## Bug Fix Template
Task: [Describe bug] File: [path] Lines: [range]
Constraints: Do NOT change [signatures, imports]
Success criteria: [How to verify]

## Feature Addition Template
Plan first: Yes/No Constraints: Do NOT break [existing functionality]

## Code Review Template
Focus: [Security, Performance, Edge cases]
Ignore: [Style, formatting]
```

Success Criteria: Constrained prompt · Plan first · Negative constraints · Template created

Module Summary

Lesson	Topic	Key Takeaway
3.1	Ask vs Agent Mode	Use Ask for questions, Agent for action
3.2	Browser Tool	Agent can see live pages and console
3.3	Terminal Tool	Agent can run commands and react
3.4	Effective Prompting	Boundaries prevent scope creep

Quick Reference Card

MODES:

ASK MODE → Read-only | Questions, learning

AGENT MODE → Full tools | Implementation, debugging

TOOLS:

BROWSER → Navigate, read, console, screenshot

TERMINAL → Execute commands, read output

FILE → Read, edit, create, delete

SEARCH → Code search, symbol lookup

PROMPTING:

- Be specific
- Define boundaries
- Plan first for complex tasks
- Specify output format
- Define success criteria

Customizing Cursor for Your Team

Module 4 · Day 1 (Hands-On + Walkthrough)

Cursor Training Program · ~60 min

Module Overview

Aspect	Details
Duration	~60 minutes
Format	Hands-on exercise + walkthrough
Prerequisites	Modules 1–3 completed, team repository access, Cursor installed
Module Goal	Customize Cursor for team workflows with rules, skills, MCP, and subagents

Learning Objectives

By the end of this module, participants will be able to:

- Create Rules that encode team conventions and guardrails
- Write Repository Instructions for lightweight project guidance
- Build and invoke reusable Skills for specialized workflows
- Connect external tools via MCP and create slash workflows
- Understand when and how to use Subagents for delegation

Agenda

Lesson	Topic	Time
4.1	Creating a Rule	20 min
4.2	Repository Instructions	13 min
4.3	Creating and Invoking a Skill	20 min
4.4	MCP, Hooks, and Slash Workflows	10 min
4.5	Subagents	6 min

Lesson 4.1

Creating a Rule

Concept · 8 min · Exercise · 12 min

What Are Rules?

Rules are Markdown files (`.cursor/rules/*.mdc`) with persistent instructions the agent automatically applies.

Rule Type	Scope	When Applied	Example
Global	All projects	Always	"Use American English spelling"
Project	Specific repo	When opening that project	"Run <code>make test</code> before suggesting changes"
File pattern	Matching files	When editing those files	"For <code>*.py</code> files, use type hints"
User	Your account	Always across all projects	"Explain like I'm a junior developer"

Rule Structure

```
---

description: Brief description of what this rule does
globs: *.py, src/**/*.js
alwaysApply: true

---

# Rule Title

## Instructions for the Agent
Write your instructions here in natural language.

## Examples
Good: ... Bad: ...
```

Windows Exercise Environment

All exercises in this module assume **Windows 10/11** with Cursor installed.

Terminal	Use when	Open in Cursor
PowerShell	Default — Python, Git, <code>curl.exe</code> , npm, Cursor CLI (<code>agent</code>)	<code>Ctrl+`</code> → PowerShell
Git Bash	Bash syntax, <code>export VAR=...</code> , shell scripts ending in <code>.sh</code>	Terminal menu → Git Bash
Command Prompt	Legacy <code>.bat</code> files only	Terminal menu → Command Prompt
Ubuntu (WSL)	Linux-only tools or native bash without Git Bash	Terminal menu → Ubuntu (WSL)

Cursor Agent panel (`Ctrl+L`) is for natural-language prompts — not a shell.

Set default profile: Settings → `terminal.integrated.defaultProfile.windows` → **PowerShell**

Exercise 4.1 — Step 1: Setup

Platform: Windows 10/11 · **PowerShell** `Ctrl+`` (Git Bash/WSL for `.sh` scripts)

```
mkdir -p .cursor/rules
```

Create **coding standards** rule at `.cursor/rules/coding-standards.mdc`:

```
globs: **/*.{js,ts,py} | alwaysApply: true
```

```
Python: type hints, Black (88 chars), Google docstrings  
JS/TS: const over let, arrow functions, optional chaining  
General: no commented-out code, no console.log in prod
```

Exercise 4.1 — Build & Test Rule

Platform: Windows 10/11 · Prompts → **Agent panel** **Ctrl+L** · Diffs → **Editor**

Create `.cursor/rules/build-and-test.mdc`:

```
Before changes: git status, git diff  
After changes: make test / pytest / npm test → make lint  
Do NOT suggest changes that break tests or need undocumented API keys
```

Create `.cursor/rules/security.mdc`:

```
Never: hardcoded secrets, eval() on user input, SQL concatenation  
Always: input validation, rate limiting, HTTPS, safe error messages  
Flag: exec/eval with user input, password/secret in variable names
```

Exercise 4.1 — Test & File-Specific Rules

Platform: Windows 10/11 · Prompts → **Agent panel** **Ctrl+L** · Diffs → **Editor**

Step 5: Verify rules are applied: **Where:** **Cursor Agent panel** — **Ctrl+L**

```
Based on the project rules, what are the coding standards I should follow?  
What are the security guardrails?
```

Exercise 4.1 — Test & File-Specific Rules (Part 2)

Step 6: Create `.cursor/rules/react-components.mdc` for `**/*.jsx`, `**/*.tsx`: **Where:** **Cursor Agent panel** — **Ctrl+L**

- Component structure, naming (PascalCase, handleSubmit)
- Performance: React.memo, useCallback, useMemo
- Accessibility: keyboard nav, alt text, semantic HTML

Success Criteria: Created rules directory · coding, build, security rules · verified application

Lesson 4.2

Repository Instructions

Concept · 5 min · Exercise · 8 min

Rules vs. Repository Instructions

Aspect	Rules	Repository Instructions
Location	<code>.cursor/rules/*.mdc</code>	<code>.cursor/repository-instructions.md</code>
Complexity	Multiple files, scoped	Single file, global
Granularity	Per-file patterns	Entire repository
Use case	Detailed standards	High-level project overview

Repository Instructions Structure

```
# Repository Instructions for [Project Name]

## Project Purpose
## Key Technologies
## Architecture Overview
## Important Conventions
## Common Tasks (make dev, make test, make build)
## External Dependencies
## Contact
```

Exercise 4.2 — Create Instructions

Platform: Windows 10/11 · Prompts → **Agent panel** `Ctrl+L` · Diffs → **Editor**

Create `.cursor/repository-instructions.md`:

```
Project: Task Manager API (FastAPI + PostgreSQL + JWT)

Architecture: api/ → services/ → repositories/ → models/
Conventions: dependency injection, async/await, type hints required

Run dev:  uvicorn src.main:app --reload
Run tests: pytest -v
Migrations: alembic upgrade head
```

Exercise 4.2 — Verify & Maintain

Platform: Windows 10/11 · Prompts → **Agent panel** `Ctrl+L` · Diffs → **Editor**

Step 2: Ask the Agent: **Where: Cursor Agent panel** — `Ctrl+L` (or `Ctrl+I` for inline Agent)

```
What are the key technologies used in this project?
How do I run the tests?
```

Exercise 4.2 — Verify & Maintain (Part 2)

Step 3: Update instructions when: **Where: Cursor Agent panel** — `Ctrl+L`

- New team members join → add contact info
- Architecture changes → update structure
- New dependencies or common issues discovered

Success Criteria: Created instructions · included purpose, stack, commands · verified agent access

Lesson 4.3

Creating and Invoking a Skill

Concept · 8 min · Exercise · 12 min

What Is a Skill?

A reusable, specialized workflow the agent loads and follows — a **"prompt template with memory."**

```
.cursor/skills/
├─ skill-name/
│   ├── SKILL.md           # Main instructions (required)
│   ├── scripts/           # Optional helper scripts
│   ├── references/        # Optional reference docs
│   └─ templates/         # Optional output templates
```

Use Case	Example Skill
Frequent task	"PR Review"
Complex workflow	"Onboarding"
Domain-specific	"Security Audit"

Use Case	Example Skill
Documentation	"Generate API Docs"

Exercise 4.3 — PR Review Skill

Platform: Windows 10/11 · Prompts → **Agent panel** **Ctrl+L** · Diffs → **Editor**

Create `.cursor/skills/pr-review/SKILL.md`:

```
name: pr-review
description: Review a PR for code quality, security, and team standards

Step 1: Fetch diff (git fetch + git diff main...FETCH_HEAD)
Step 2: Review – code quality, security, testing, docs, style
Step 3: Output formatted review with Critical / Warning / Suggestion
Verdict: APPROVE / REQUEST CHANGES / COMMENT
```

Exercise 4.3 — Security Audit Skill

Platform: Windows 10/11 · Prompts → **Agent panel** **Ctrl+L** · Diffs → **Editor**

Create `.cursor/skills/security-audit/SKILL.md`:

```
Scan for:
  Critical: hardcoded secrets, SQL injection, command injection, eval()
  Medium:   no input validation, weak crypto, missing CSRF
  Low:      debug endpoints, verbose errors, outdated deps

Output: report with line numbers, fix suggestions, overall risk rating
```

Exercise 4.3 — Invoke Skills

Platform: Windows 10/11 · Prompts → **Agent panel** **Ctrl+L** · Diffs → **Editor**

Step 4: Invoke via slash command: **Where:** **Cursor Agent panel** — **Ctrl+L**

```
/pr-review PR #42
/pr-review feature/payment-integration
```

Exercise 4.3 — Invoke Skills (Part 2)

Step 5: List available skills: **Where:** **Cursor Agent panel** — **Ctrl+L**

What skills are available in this project?

Exercise 4.3 — Invoke Skills (Part 3)

Step 6: Create **Onboarding** skill — generates setup checklist from repo instructions **Where:** **Cursor Agent panel** — **Ctrl+L**

Success Criteria: Created skills · built PR Review + Security Audit · invoked via slash command

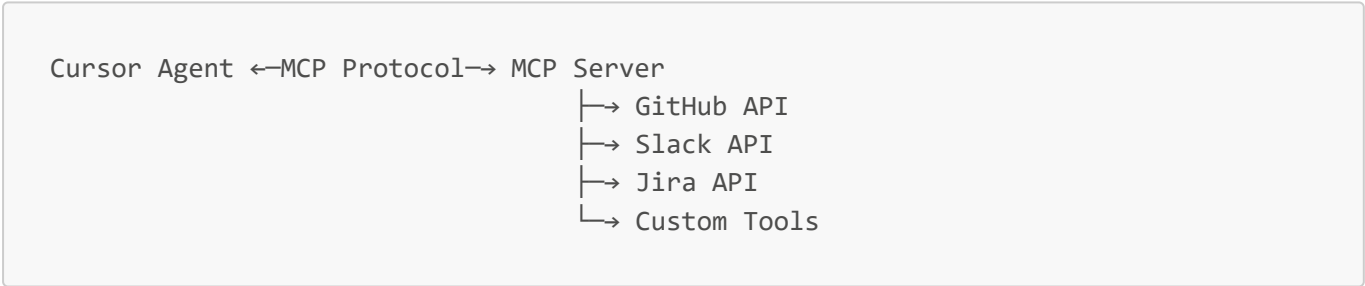
Lesson 4.4

MCP, Hooks, and Slash Workflows

Concept · 10 min · Walkthrough

What Is MCP?

MCP standardizes how AI agents discover and use external tools — **"USB port for AI."**



MCP Server	Capabilities
GitHub	Create PRs, comment on issues, fetch reviews
Slack	Send messages, read channels
Jira	Create tickets, update status
Database	Query databases, run migrations

Hooks & Slash Workflows

Hooks — scripts at specific agent workflow points:

Hook	When It Runs	Use Case
pre-tool-use	Before tool call	Validate permissions, log
post-tool-use	After tool returns	Transform results, audit

Hook	When It Runs	Use Case
pre-prompt	Before sending to model	Inject context, redact secrets
post-response	After agent responds	Format output, log

Slash workflows — team commands combining MCP, hooks, and prompts:

Command	What It Does
/onboard @newdev	Onboarding skill + GitHub issue + Slack message
/deploy staging	Tests → build → deploy → notify team
/bug-report	Analyze error → create issue → assign on-call

Walkthrough: MCP Configuration

Create `~/.cursor/mcp.json`:

```
{
  "mcpServers": {
    "github": {
      "command": "cursor-mcp-github",
      "args": ["--token", "${GITHUB_TOKEN}"]
    },
    "slack": {
      "command": "cursor-mcp-slack",
      "args": ["--token", "${SLACK_TOKEN}"]
    }
  }
}
```

```
Create a PR from feature/payment and request review from @alice
Send a message to #deploys: "Deployment starting"
```

Walkthrough: Slash Command Example

Create `.cursor/commands/deploy.md`:

```
name: deploy | arguments: environment (staging | production)

Pre-deploy: make test → make lint → git status clean
Deploy staging: git push origin main → kubectl set image
Deploy production: git tag → deploy-prod.sh (requires approval)
Post-deploy: health check → smoke tests → Slack notify
```

Usage: `/deploy staging`

Success Criteria: Understood MCP, hooks, slash commands · saw configuration examples

Lesson 4.5

Subagents

Concept · 6 min · Walkthrough

What Are Subagents?

Independent agent instances for specialized tasks — own context, tools, and instructions — then report back to the main agent.

```
Main Agent
├── Subagent: Security Scanner → "2 critical issues found"
├── Subagent: Documentation Generator → "Updated README.md"
└── Subagent: Test Writer → "Added 5 test cases"
```

When to Use Subagents

Scenario	Why Subagent	Example
Parallel work	Multiple tasks simultaneously	Scan security AND generate docs
Isolation	Separate context	Analyze large file independently
Specialization	Different instructions	Security expert vs. UI designer
Sandboxing	Limit tool access	Read-only subagent for unknown code

Subagent vs. Tool vs. Skill

Concept	Best for
Tool	Single action (read file, run command)

Concept	Best for
Skill	Multi-step workflow, same context
Subagent	Parallel, isolated, specialized work

Walkthrough: Subagents in Action

Task: "Review codebase for security issues and generate API documentation"

Without subagents: Mixed context, sequential, slower

With subagents (parallel):

```
→ Subagent 1 (Security): scan all files, read-only
→ Subagent 2 (Docs): generate OpenAPI spec, write to docs/

Main Agent combines: security report + openapi.yaml
```

Invoke:

```
Spawn a security subagent to audit src/auth/ independently.
Meanwhile, I'll work on the frontend.
```

Success Criteria: Understood concept · parallel execution · recognized templates

Module Summary

Lesson	Topic	Key Output
4.1	Creating a Rule	<code>.cursor/rules/*.mdc</code> files
4.2	Repository Instructions	<code>.cursor/repository-instructions.md</code>
4.3	Creating Skills	<code>.cursor/skills/*/SKILL.md</code>
4.4	MCP & Slash Commands	MCP config, slash commands
4.5	Subagents	Understanding of delegation

Quick Reference Card

```
RULES           → .cursor/rules/*.mdc (standards, security, build)
REPO INSTR.     → .cursor/repository-instructions.md (project overview)
SKILLS          → .cursor/skills/*/SKILL.md (invoke with /skill-name)
MCP             → ~/.cursor/mcp.json (GitHub, Slack, Jira, custom)
```

SLASH CMDS	→ .cursor/commands/*.md (/deploy, /onboard, /trriage)
SUBAGENTS	→ Parallel, isolated, specialized execution

Cursor CLI and Local Automation

Module 5 · Day 1 (Hands-On)

Cursor Training Program · ~60 min

Module Overview

Aspect	Details
Duration	~60 minutes
Format	Hands-on exercise
Prerequisites	Cursor CLI installed, terminal access, Modules 1–4 completed
Module Goal	Master the Cursor CLI for terminal-based AI workflows and automation

Learning Objectives

By the end of this module, participants will be able to:

- Use the Cursor CLI in interactive mode for real-time AI collaboration
- Run one-shot CLI commands for scripting and CI/CD integration
- Hand off local sessions to Cloud Agents for remote execution
- List, resume, and manage concurrent sessions effectively

Agenda

Lesson	Topic	Time
5.1	Interactive CLI	20 min
5.2	One-Shot CLI	20 min
5.3	Cloud Handoff	18 min
5.4	Listing and Resuming Sessions	20 min

Lesson 5.1

Interactive CLI

Concept · 8 min · Exercise · 12 min

What Is the Cursor CLI?

The Cursor CLI brings AI-powered coding directly to your command line.

- Start AI sessions from your terminal
- Get code assistance without leaving your workflow
- Automate coding tasks with scripts
- Integrate AI into existing CLI tools

Primary command: `agent` (main entry point)

Interactive Mode Commands

Command	Purpose
<code>/model</code>	Switch between AI models interactively
<code>/compress</code>	Summarize conversation, free up context window
<code>/rules</code>	Create and edit rules directly from CLI
<code>/commands</code>	Create and modify custom commands
<code>/mcp enable/disable</code>	Manage MCP servers
<code>/usage</code>	View Cursor usage stats
<code>/about</code>	View environment and CLI configuration
<code>/resume</code>	View and resume previous sessions

Windows Exercise Environment

All exercises in this module assume **Windows 10/11** with Cursor installed.

Terminal	Use when	Open in Cursor
PowerShell	Default — Python, Git, <code>curl.exe</code> , npm, Cursor CLI (<code>agent</code>)	<code>Ctrl+`</code> → PowerShell
Git Bash	Bash syntax, <code>export VAR=...</code> , shell scripts ending in <code>.sh</code>	Terminal menu → Git Bash
Command Prompt	Legacy <code>.bat</code> files only	Terminal menu → Command Prompt
Ubuntu (WSL)	Linux-only tools or native bash without Git Bash	Terminal menu → Ubuntu (WSL)

Cursor Agent panel (`Ctrl+L`) is for natural-language prompts — not a shell.

Set default profile: Settings → `terminal.integrated.defaultProfile.windows` → **PowerShell**

Exercise 5.1 — Steps 1–2

Platform: Windows 10/11 · **PowerShell** `Ctrl+`` (Git Bash/WSL for `.sh` scripts)

Step 1: Start an interactive session **Terminal: PowerShell** — unless step notes Git Bash or WSL

```
agent
agent "Help me understand the current codebase structure"
```

Exercise 5.1 — Steps 1–2 (Part 2)

Step 2: Navigate the session (inside the running `agent` session — same terminal window) — unless step notes Git Bash or WSL

- Type prompts naturally
 - `Shift+Enter` — new line without submitting
 - `Enter` — submit prompt
 - `Ctrl+D` twice — exit
-

Exercise 5.1 — Steps 3–5

Platform: Windows 10/11 · **PowerShell** `Ctrl+`` (Git Bash/WSL for `.sh` scripts)

Step 3: Switch models: **Terminal: PowerShell** — unless step notes Git Bash or WSL

```
/model
# Or list models outside session:
agent --list-models
```

Exercise 5.1 — Steps 3–5 (Part 2)

Step 4: Ask Mode (read-only): **Where: Cursor Agent panel** — `Ctrl+L` (or `Ctrl+I` for inline Agent)

```
agent --mode=ask "What does this project's main function do?"
# Or inside session: /ask
```

Exercise 5.1 — Steps 3–5 (Part 3)

Step 5: Plan Mode: **Where: Cursor Agent panel** — `Ctrl+L` (or `Ctrl+I` for inline Agent)

```
agent --mode=plan "Add user authentication to this API"
```

Exercise 5.1 — Steps 6–7

Platform: Windows 10/11 · **PowerShell** `Ctrl+`` (Git Bash/WSL for `.sh` scripts)

Step 6: Configure status line: **Terminal: PowerShell** — unless step notes Git Bash or WSL

```
npx -y cursor-statusline
# Shows: [model: claude-4.5-sonnet] [~/project] [main] [ctx: 45k/200k]
```

Exercise 5.1 — Steps 6–7 (Part 2)

Step 7: Terminal key bindings: **Terminal: PowerShell** — unless step notes Git Bash or WSL

```
agent /setup-terminal
```

Lesson 5.2

One-Shot CLI

Concept · 8 min · Exercise · 12 min

One-Shot Command Structure

```
agent "your prompt here"           # Basic one-shot
agent --mode=ask "question about code" # Read-only
agent --model claude-4.5-sonnet "task"  # Specific model
agent --non-interactive "run this task"  # No prompts, just output
```

"Perfect for automation, CI/CD pipelines, and batch operations."

Use Cases for One-Shot CLI

Use Case	Example
Code generation	agent "Create a React component for a login form"

Use Case	Example
Documentation	<code>agent "Generate JSDoc comments for src/api.js"</code>
CI/CD tasks	<code>agent "Review this PR diff for security issues"</code>
Batch processing	Loop through files with <code>agent</code> commands
Pre-commit hooks	<code>agent --mode=ask "Check for console.log statements"</code>

Exercise 5.2 — Steps 1–2

Platform: Windows 10/11 · **PowerShell** `Ctrl+`` (Git Bash/WSL for `.sh` scripts)

Step 1: Basic one-shot commands: **Terminal: PowerShell** — unless step notes Git Bash or WSL

```
agent "What is the difference between let and const in JavaScript?"
agent "Write a bash function that checks if a port is in use"
agent --mode=ask "Explain the git rebase command with examples"
```

Exercise 5.2 — Steps 1–2 (Part 2)

Step 2: Specify models: **Terminal: PowerShell** — unless step notes Git Bash or WSL

```
agent --model gpt-5-mini "What does this command do: ls -la | grep .txt"
agent --model claude-4.5-opus "Design a database schema for a task management system"
```

Exercise 5.2 — Scriptable Code Reviewer

Platform: Windows 10/11 · **PowerShell** for API · `$env:VAR` · `curl.exe`

Create `bin/ai-review.sh`:

```
#!/bin/bash
STAGED_FILES=$(git diff --cached --name-only | tr '\n' ', ')

agent --mode=ask "Review these staged files for common issues:
Files: $STAGED_FILES
Check for: debugging statements, unused imports,
security issues, missing error handling. Be concise."
```

PowerShell (Windows): Same steps in **PowerShell** — use `$env:NAME = "value"` instead of `export`, and `curl.exe` instead of `curl`.

Exercise 5.2 — Batch & Git Hooks

Platform: Windows 10/11 · **PowerShell** for API · `$env:VAR` · `curl.exe`

Step 4: Batch process files: **Terminal: PowerShell** — unless step notes Git Bash or WSL

```
for file in src/**/*.py; do
    agent --mode=ask --non-interactive \
        "Summarize this Python file in one sentence: $(head -50 $file)"
done
```

Exercise 5.2 — Batch & Git Hooks (Part 2)

Step 5: Pre-commit hook — review staged diff for secrets, debug statements, merge markers **Where:** **Cursor Agent panel** — `Ctrl+L` (or `Ctrl+I` for inline Agent)

Step 6: CI/CD — analyze test output and suggest fixes for failures **Terminal: PowerShell** — clone/open repo, then continue in Agent panel

Success Criteria: Ran one-shots · specified models · created reviewer script · understood CI/CD use

Lesson 5.3

Cloud Handoff

Concept · 8 min · Exercise · 10 min

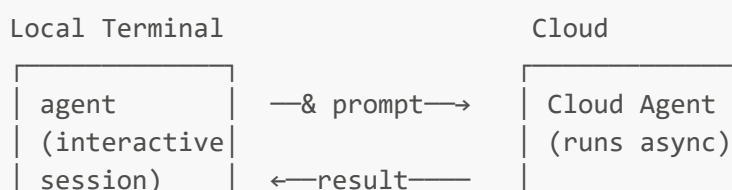
What Is Cloud Handoff?

Send a local conversation to a Cloud Agent:

- Continue from web or mobile (cursor.com/agents)
- Let the agent run long tasks while you're away
- Resume the session later from any device

The & prefix: Prepend any message with `&` to send it to the cloud.

Cloud Handoff Flow



↓
cursor.com/agents

Exercise 5.3 — Steps 1–3

Platform: Windows 10/11 · **PowerShell** `Ctrl+`` (Git Bash/WSL for `.sh` scripts)

Step 1: Start local session and hand off: **Terminal: PowerShell** — unless step notes Git Bash or WSL

```
agent  
& "Analyze the entire codebase and create a dependency graph."
```

Exercise 5.3 — Steps 1–3 (Part 2)

Step 2: Verify handoff: **Terminal: PowerShell** — unless step notes Git Bash or WSL

```
🚀 Handing off to Cloud Agent...  
✅ Session running at: https://cursor.com/agents/[agent-id]
```

Exercise 5.3 — Steps 1–3 (Part 3)

Step 3: Check status via browser or CLI **Where: Web browser** — Edge or Chrome

Exercise 5.3 — Steps 4–6

Platform: Windows 10/11 · **PowerShell** `Ctrl+`` (Git Bash/WSL for `.sh` scripts)

Step 4: Push existing conversation: **Terminal: PowerShell** — unless step notes Git Bash or WSL

```
& "Continue this conversation in the cloud. I need to log off."
```

Exercise 5.3 — Steps 4–6 (Part 2)

Step 5: Long-running task: **Terminal: PowerShell** — unless step notes Git Bash or WSL

```
agent "& Refactor the auth module to use JWT. Update all tests and docs."
```

Exercise 5.3 — Steps 4–6 (Part 3)

Step 6: Resume later: **Terminal: PowerShell** — unless step notes Git Bash or WSL

```
agent --resume [agent-id-from-cloud]
```

Cloud Handoff Best Practices

When to Use	When Not to Use
Long-running tasks (>5 min)	Quick questions
When you need to close laptop	Interactive debugging
Overnight batch processing	Tasks needing terminal access
Parallel work streams	Security-sensitive code (local only)

Success Criteria: Sent & message · verified cloud agent · accessed via web

Lesson 5.4

Listing and Resuming Sessions

Concept · 8 min · Exercise · 12 min

Session Management Commands

Command	Purpose
/resume	List all previous sessions and resume one
agent --resume [id]	Resume a specific session by ID
agent --list	List available sessions (alternative)

Tip: Name sessions with the first message:

```
agent "Just say one word: auth-refactor"  
# Session named "auth-refactor Agent"
```

Exercise 5.4 — Steps 1–2

Platform: Windows 10/11 · **PowerShell** **Ctrl+`** (Git Bash/WSL for **.sh** scripts)

Step 1: Create multiple named sessions: **Terminal: PowerShell** — unless step notes Git Bash or WSL

```
agent "Just say one word: frontend-cleanup" # do work, exit
agent "Just say one word: db-optimization" # do work, exit
agent "Just say one word: docs-update"
```

Exercise 5.4 — Steps 1–2 (Part 2)

Step 2: List all sessions: **Terminal: PowerShell** — unless step notes Git Bash or WSL

```
/resume
# 1. frontend-cleanup Agent (2 hours ago)
# 2. db-optimization Agent (1 hour ago)
# 3. docs-update Agent (30 minutes ago)
```

Exercise 5.4 — Steps 3–5

Platform: Windows 10/11 · **PowerShell** `Ctrl+`` (Git Bash/WSL for `.sh` scripts)

Step 3: Resume by ID: **Terminal: PowerShell** — unless step notes Git Bash or WSL

```
agent --resume abc123-def456-ghi789
```

Exercise 5.4 — Steps 3–5 (Part 2)

Step 4: Concurrent sessions in different terminals: **Where: Cursor Agent panel** — `Ctrl+L` (or `Ctrl+I` for inline Agent)

```
# Terminal 1: agent --resume frontend-cleanup
# Terminal 2: agent --resume db-optimization
```

Exercise 5.4 — Steps 3–5 (Part 3)

Step 5: Context management: **Terminal: PowerShell** — unless step notes Git Bash or WSL

```
/compress # Summarize conversation, free context window
```

Exercise 5.4 — Steps 6–7 & Best Practices

Platform: Windows 10/11 · Prompts → **Agent panel** `Ctrl+L` · Diffs → **Editor**

Step 6: Export session summary as markdown **Where:** **Cursor Agent panel** — `Ctrl+L`

Step 7: Create `bin/cursor-sessions.sh` to list and manage sessions **Where:** **Cursor Agent panel** — `Ctrl+L`

Naming: Use `[area]-[task]` format (e.g., `api-auth-fix`)

Context: Use `/compress` on long sessions · cloud handoff for very long tasks

Cleanup: Sessions persist indefinitely — manually complete or discard finished ones

Success Criteria: Created named sessions · listed with `/resume` · resumed · used `/compress`

Module Summary

Lesson	Topic	Key Skill
5.1	Interactive CLI	Real-time terminal AI
5.2	One-Shot CLI	Scripting & automation
5.3	Cloud Handoff	Remote/long-running tasks
5.4	Session Management	Concurrent work handling

Quick Reference Card

```
BASIC:
  agent                Start interactive session
  agent "prompt"        One-shot command
  agent --mode=ask       Read-only mode
  agent --mode=plan      Plan before code
  agent --model <name>   Specify model

IN SESSION:  /model  /compress  /rules  /commands  /resume  /usage

CLOUD:  & "message" → cursor.com/agents

KEYS:  Shift+Enter (new line) | Ctrl+D twice (exit)
```

Day 2

Cloud Agents, APIs & Analytics

Cloud Agents in the UI

Module 6 · Day 2 (Hands-On + Demonstration)

Cursor Training Program · ~90 min

Module Overview

Aspect	Details
Duration	~90 minutes
Format	Hands-on exercise + demonstration
Prerequisites	Cursor account, GitHub repository access, Modules 1–5 completed
Module Goal	Master Cloud Agents UI for remote execution, artifact collection, and messaging integrations

Learning Objectives

By the end of this module, participants will be able to:

- Launch and monitor Cloud Agents from the Cursor UI
- Collect and download artifacts from completed cloud runs
- Trigger Cloud Agents from messaging platforms (Slack, Discord)
- Manage cloud agent history and settings

Agenda

Lesson	Topic	Time
6.1	Launching a Cloud Agent	25 min
6.2	Cloud Agent Artifacts	23 min
6.3	Cloud Agents from Messaging Platforms	20 min

Lesson 6.1

Launching a Cloud Agent

Concept · 10 min · Exercise · 15 min

Cloud Agents vs. Local Agent

Aspect	Local Agent	Cloud Agent
Runs on	Your machine	Cursor's infrastructure
Persistence	Ends when you quit	Continues indefinitely
Access	Local only	Web, mobile, API
Terminal access	Your terminal	Simulated/scripted
File access	Local files	GitHub repos only
Best for	Interactive work	Batch, scheduled, hands-off

When to Use Cloud Agents

Good for:

- Long-running tasks (> 10 min) · Scheduled jobs
- Tasks while offline · Parallel execution
- Team-accessible results (share agent URL)

Bad for:

- Interactive debugging · Local-only files
- Security-sensitive code · Quick questions

Accessing Cloud Agents UI

Method	Steps
From Cursor Editor	View → Cloud Agents (or cloud icon in sidebar)
From Web	https://cursor.com/agents
From Mobile	cursor.com/agents (responsive web)

Cloud Agent Dashboard

Active (2)

 security-audit-2024

running • 12 min elapsed

 doc-generator

running • 3 min elapsed

Completed (4)

 pr-review-42

FINISHED • 2 artifacts

 test-suite

FINISHED • 1 artifact

Failed (1)

 deploy-staging

ERROR • Auth token expired

Windows Exercise Environment

All exercises in this module assume **Windows 10/11** with Cursor installed.

Terminal	Use when	Open in Cursor
PowerShell	Default — Python, Git, <code>curl.exe</code> , npm, Cursor CLI (<code>agent</code>)	<code>Ctrl+`</code> → PowerShell
Git Bash	Bash syntax, <code>export VAR=...</code> , shell scripts ending in <code>.sh</code>	Terminal menu → Git Bash
Command Prompt	Legacy <code>.bat</code> files only	Terminal menu → Command Prompt
Ubuntu (WSL)	Linux-only tools or native bash without Git Bash	Terminal menu → Ubuntu (WSL)

Cursor Agent panel (`Ctrl+L`) is for natural-language prompts — not a shell.

Set default profile: Settings → `terminal.integrated.defaultProfile.windows` → **PowerShell**

Exercise 6.1 — Steps 1–2

Platform: Windows 10/11 • **PowerShell** `Ctrl+`` (Git Bash/WSL for `.sh` scripts)

Step 1: Navigate to Cloud Agents **Terminal: PowerShell** — `Ctrl+`` in Cursor

```
# Cursor Editor: cloud icon or View → Cloud Agents
open https://cursor.com/agents
```

Exercise 6.1 — Steps 1–2 (Part 2)

Step 2: Click "+ New" and fill out: **Terminal: PowerShell** — unless step notes Git Bash or WSL

```
Repository: https://github.com/YOUR_ORG/YOUR_REPO
Branch: main
Prompt: Read README and main source files. Summarize:
  - What this project does
  - Key dependencies • How to run locally • Common issues
Model: claude-4.6-sonnet
Auto-create PR: ☐
```

Exercise 6.1 — Steps 3–4

Platform: Windows 10/11 · Prompts → **Agent panel** `Ctrl+L` · Diffs → **Editor**

Step 3: Monitor live log in real time: **Where:** **Cursor Agent panel** — `Ctrl+L`

```
[10:45:01] Agent starting...
[10:45:02] Cloning repository...
[10:45:15] Repository cloned
[10:45:16] Reading README.md
[10:45:40] Generating summary...
```

Exercise 6.1 — Steps 3–4 (Part 2)

Step 4: Configure settings (gear icon): **Where:** **Cursor Agent panel** — `Ctrl+L`

Setting	Purpose
Default Model	Preferred model for new agents
Auto-create PR	Create PRs on completion
Notification Email	Completion notifications
Webhook URL	POST completion events
Max Run Time	5 min – 24 hrs

Exercise 6.1 — Steps 5–6

Platform: Windows 10/11 · Prompts → **Agent panel** `Ctrl+L` · Diffs → **Editor**

Step 5: Launch with PR creation: **Where:** **Cursor Agent panel** — `Ctrl+L`

```
Prompt: Add CONTRIBUTING.md with dev setup, tests, PR process, code style
Auto-create PR: ☒ Yes
Branch prefix: docs/contributing
```

Exercise 6.1 — Steps 5–6 (Part 2)

Step 6: Share agent URL with team: **Where:** **Cursor Agent panel** — `Ctrl+L`

```
https://cursor.com/agents/agt_abc123def456
```

Lesson 6.2

Cloud Agent Artifacts

Concept · 8 min · Exercise · 15 min

Types of Artifacts

Artifact Type	Examples
Log files	<code>agent.log</code> , <code>debug.log</code>
Code files	<code>*.py</code> , <code>*.js</code> , <code>*.html</code>
Documents	<code>*.md</code> , <code>*.txt</code> , <code>*.json</code>
Images	<code>*.png</code> , <code>*.jpg</code> , <code>*.svg</code>
Archives	<code>*.zip</code> , <code>*.tar.gz</code>
Test results	<code>junit.xml</code> , <code>coverage.json</code>

"Files produced by the agent that you can download or view in the UI."

Artifact Storage

- Stored for **30 days**
- Multiple artifacts per agent
- Download URLs expire after **15 minutes**
- Max **100MB** per file · **1GB** total per agent

Exercise 6.2 — Steps 1–2

Platform: Windows 10/11 · Prompts → **Agent panel** `Ctrl+L` · Diffs → **Editor**

Step 1: Launch agent that generates artifacts: **Where:** **Cursor Agent panel** — `Ctrl+L`

Generate:

1. `api_documentation.md` – OpenAPI-style docs for all endpoints

2. `test_report.json` – test suite summary

3. `screenshot.png` – main UI screenshot (if applicable)

4. `dependencies.txt` – all packages and versions

Place all in `artifacts/` directory.

Exercise 6.2 — Steps 1–2 (Part 2)

Step 2: After completion, view artifact list in UI with Download buttons and **Download All (zip)** Where: **Cursor Agent panel** — **Ctrl+L**

Exercise 6.2 — Steps 3–5

Platform: Windows 10/11 · Agent → **Ctrl+L** · Shell → **PowerShell** · Browser for dashboards

Step 3: Download individual artifacts **Where: Cursor Agent panel** — **Ctrl+L**

Step 4: Download all as zip **Where: Cursor Agent panel** — **Ctrl+L**

Exercise 6.2 — Steps 3–5 (Part 2)

Step 5: Preview in browser: **Where: Web browser** — Edge or Chrome

- Markdown → rendered HTML
 - Images → inline preview
 - JSON → formatted tree view
-

Exercise 6.2 — API Access

Platform: Windows 10/11 · **PowerShell** for API · **\$env:VAR** · **curl.exe**

```
# List artifacts
curl -s -u "$CURSOR_USER_API_KEY:" \
  "https://api.cursor.com/v1/agents/$AGENT_ID/artifacts" | jq '.'

# Download specific artifact
DOWNLOAD_URL=$(curl -s -u "$CURSOR_USER_API_KEY:" \
  ".../artifacts/download?path=artifacts/report.md" | jq -r '.url')
curl -L -o report.md "$DOWNLOAD_URL"
```

PowerShell (Windows): Same steps in **PowerShell** — use **\$env:NAME = "value"** instead of **export**, and **curl.exe** instead of **curl**.

Create **bin/process-artifacts.sh** to batch-download all artifacts for an agent ID.

Exercise 6.2 — CI/CD Integration

Platform: Windows 10/11 · **PowerShell** for API · **\$env:VAR** · **curl.exe**

```
# GitHub Actions – download test results from completed agent
- name: Download Cloud Agent artifacts
  run: |
```

```
curl -s -u "${{ secrets.CURSOR_API_KEY }}" \
  ".../artifacts/download?path=test_results.xml" > test_results.xml
```

Success Criteria: Generated artifacts · downloaded single + zip · accessed via API

Lesson 6.3

Cloud Agents from Messaging Platforms

Concept · 10 min · Demonstration

Supported Integrations

Platform	Capabilities	Setup
Slack	Command triggering, notifications	Medium (Slack app)
Discord	Command triggering, webhook responses	Medium (Bot token)
Teams	Webhook integration	Medium (Webhook)
Generic Webhook	POST-triggered agents	Low (any platform)

Messaging Integration Architecture



Demo: Slack Integration

Step 1: Create Slack App at api.slack.com

Step 2: Configure slash command:

```
Command: /cursor
Request URL: https://your-server.com/webhook/slack-cursor
```

Usage Hint: [prompt or command]

Step 3: Deploy webhook receiver (Flask/Python) that:

- Parses Slack command → launches Cloud Agent via API
- Acknowledges immediately with agent URL
- Posts completion summary when webhook fires

Demo: Slack Usage

In Slack:

```
/cursor Review the recent commits and summarize what changed
```

Response:

```
🤖 Launching Cloud Agent `agt_abc123`  
Watch progress: https://cursor.com/agents/agt_abc123  
  
✅ Cloud Agent Complete!  
Summary: 3 commits – fixed login bug, added tests, updated README.  
PR: https://github.com/your-org/your-repo/pull/43
```

Demo: Discord Integration

```
@bot.command(name='cursor')  
async def cursor_command(ctx, *, prompt):  
    response = requests.post("https://api.cursor.com/v1/agents", ...)  
    await ctx.send(f"✅ Agent launched: https://cursor.com/agents/{agent_id}")
```

Usage: `!cursor` Add error handling to all API endpoints

Generic Webhook & Notifications

Any HTTP POST can trigger agents:

```
curl -X POST https://your-server.com/trigger-agent \  
-H "Content-Type: application/json" \  
-d '{"prompt": "Run the weekly security scan", "repo": "..."}'
```

Use cases: GitHub webhook on PR · Cron jobs · CI/CD post-deploy · Internal dashboard

Status notifications: configure `notifyOnStart`, `notifyOnComplete`, `notifyOnError`

Success Criteria: Understood architecture · saw Slack/Discord demos · webhook triggering

Module Summary

Lesson	Topic	Key Skill
6.1	Launching Cloud Agents	Remote execution
6.2	Cloud Agent Artifacts	Output collection
6.3	Messaging Integrations	Chat-triggered agents

Quick Reference Card

UI ACCESS:

Cursor Editor: View → Cloud Agents

Web: <https://cursor.com/agents>

LAUNCH: + New → repo/branch → prompt → model → Launch Agent

ARTIFACTS: View in detail page · Download single/zip · API (30-day storage)

MESSAGING:

/cursor [prompt] in Slack

!cursor [prompt] in Discord

Webhook POST to trigger endpoint

Cursor API Foundations

Module 7 · Day 2 (Concept + Hands-On)

Cursor Training Program · ~60 min

Module Overview

Aspect	Details
Duration	~60 minutes
Format	Concept + hands-on exercise
Prerequisites	Cursor account, basic API familiarity, Python 3.8+ installed

Aspect	Details
Module Goal	Understand the Cursor API ecosystem, authenticate securely, handle errors, and optimize requests

Learning Objectives

By the end of this module, participants will be able to:

- Identify the five Cursor APIs and their use cases
- Generate and securely manage API keys
- Implement rate limit handling and error recovery
- Use ETag caching for efficient repeat queries
- Test authentication by listing available models

Agenda

Lesson	Topic	Time
7.1	The Cursor API Landscape	10 min
7.2	Authentication	20 min
7.3	Rate Limits and Error Handling	20 min
7.4	ETag Caching	18 min
7.5	Listing Available Models	10 min

Lesson 7.1

The Cursor API Landscape

Concept · 10 min

The Five APIs

API	Endpoint	Purpose
Chat Completions	/v1/chat/completions	OpenAI-compatible chat interface
Agents	/v1/agents	Create and manage Cloud Agents
Files	/v1/files	Upload/download files for agents
Admin	/v1/admin/*	Team management, analytics, policies
Webhooks	/v1/webhooks	Register and manage webhook endpoints

API Comparison Matrix

API	Auth Type	Rate Limit	Cost	Primary Use
Chat Completions	User or API key	Per-minute token	Pay-per-token	Direct model access
Agents	User API key	Per-minute requests	Per-run	Long-running tasks
Files	User API key	Per-minute	Storage	Context upload
Admin	Admin API key	Higher limits	Included in plan	Team management
Webhooks	User API key	Per-minute	Free	Notifications

When to Use Which API

- **Call a model directly** → Chat Completions API (OpenAI-compatible)
- **Run a long task that writes code** → Agents API
- **Manage team usage and limits** → Admin API
- **Be notified when agents complete** → Webhooks API

OpenAI Compatibility

```
from openai import OpenAI

client = OpenAI(
    base_url="https://api.cursor.com/v1",
    api_key="your-cursor-api-key"
)

response = client.chat.completions.create(
    model="claude-4.6-sonnet",
    messages=[{"role": "user", "content": "Hello!"}]
)
```

Success Criteria: Understand five APIs · select correct API · understand OpenAI compatibility

Lesson 7.2

Authentication

Concept · 8 min · Exercise · 12 min

Authentication Methods

Method	Format	When to Use
HTTP Basic	-u "api_key:"	CLI, curl, most SDKs
Bearer Token	Authorization: Bearer <key>	OAuth-style clients
User API Key	Regular key	Agents, Chat, Files APIs
Admin API Key	admin_ prefixed	Admin API only

API Key Types

User API Key

- Generated in: Cursor Settings → API Keys
- Format: `cursor_xxxxxxxxxxxx`
- Can access: Agents, Chat, Files, Webhooks

Admin API Key

- Generated in: Organization Settings → API Keys
- Format: `cursor_admin_xxxxxxxxxxxx`
- Can access: Admin API + everything User can

Security Best Practices

- Never commit API keys to git
- Use environment variables or secret managers
- Rotate keys periodically (every 90 days)
- Use different keys for dev and production
- Revoke unused keys immediately
- Use Admin API keys only when necessary
- Monitor key usage in dashboard

Windows Exercise Environment

All exercises in this module assume **Windows 10/11** with Cursor installed.

Terminal	Use when	Open in Cursor
PowerShell	Default — Python, Git, <code>curl.exe</code> , npm, Cursor CLI (<code>agent</code>)	<code>Ctrl+`</code> → PowerShell
Git Bash	Bash syntax, <code>export VAR=...</code> , shell scripts ending in <code>.sh</code>	Terminal menu → Git Bash
Command Prompt	Legacy <code>.bat</code> files only	Terminal menu → Command Prompt

Terminal	Use when	Open in Cursor
Ubuntu (WSL)	Linux-only tools or native bash without Git Bash	Terminal menu → Ubuntu (WSL)

Cursor Agent panel (**Ctrl+L**) is for natural-language prompts — not a shell.

Set default profile: Settings → `terminal.integrated.defaultProfile.windows` → **PowerShell**

Exercise 7.2 — Steps 1–3

Platform: Windows 10/11 · **PowerShell** for API · `$env:VAR` · `curl.exe`

Step 1: Generate User API Key — **Where:** **Cursor app** → **Settings** → **API Keys** → **Generate New Key** (copy the key; you will not see it again)

Exercise 7.2 — Steps 1–3 (Part 2)

Step 2: Set environment variable — **Terminal: PowerShell** (**Ctrl+`**)

```
$env:CORSOR_USER_API_KEY = "cursor_XXXXXXXXXXXX"
$env:CORSOR_USER_API_KEY
```

Exercise 7.2 — Steps 1–3 (Part 3)

Step 3: Test with curl — **Terminal: PowerShell**

```
curl.exe -s -u "$($env:CORSOR_USER_API_KEY):" `
https://api.cursor.com/v1/models | Select-Object -First 20
```

Exercise 7.2 — Steps 4–5

Platform: Windows 10/11 · **PowerShell** for API · `$env:VAR` · `curl.exe`

Step 4: Test with Python requests: **Terminal: PowerShell** — save as `test_models.py`, then `python test_models.py` — **Ctrl+L**

```
response = requests.get(
    "https://api.cursor.com/v1/models",
    auth=(API_KEY, "") # Empty password
)
```

Exercise 7.2 — Steps 4–5 (Part 2)

Step 5: Test with OpenAI SDK: **Terminal: PowerShell** — `python test_openai_sdk.py` — `Ctrl+L`

```
client = OpenAI(base_url="https://api.cursor.com/v1", api_key=API_KEY)
response = client.chat.completions.create(
    model="gpt-5-mini",
    messages=[{"role": "user", "content": "Say 'API works!'"}],
    max_tokens=10
)
```

Exercise 7.2 — Steps 6–7

Platform: Windows 10/11 · **PowerShell** for API · `$env:VAR` · `curl.exe`

Step 6: Generate and test Admin API Key: **Terminal: PowerShell** — unless step notes Git Bash or WSL

```
export CURSOR_ADMIN_API_KEY="cursor_admin_XXXXXXXXXXXX"
curl -s -u "$CURSOR_ADMIN_API_KEY:" \
  https://api.cursor.com/v1/admin/organization | jq '.'
```

Exercise 7.2 — Steps 6–7 (Part 2)

Step 7: Revoke compromised keys via API or Settings → API Keys → Revoke **Terminal: PowerShell** — unless step notes Git Bash or WSL

Success Criteria: Generated keys · tested curl, Python, OpenAI SDK · tested Admin key

Lesson 7.3

Rate Limits and Error Handling

Concept · 10 min · Exercise · 10 min

Rate Limits by API

API	Limit	Window
Chat Completions	1000 requests	per minute
Chat Completions (tokens)	500k tokens	per minute
Agents (create)	100 requests	per minute

API	Limit	Window
Admin API	500 requests	per minute
Webhooks	2000 requests	per minute

HTTP Status Codes to Handle

Code	Meaning	Action
200	Success	Process response
400	Bad Request	Fix request parameters
401	Unauthorized	Check API key
403	Forbidden	Check permissions
429	Too Many Requests	Implement backoff
500/503	Server Error	Retry with backoff

Rate Limit Headers

Header	Description	Example
X-RateLimit-Limit	Max requests per window	1000
X-RateLimit-Remaining	Requests left	942
X-RateLimit-Reset	Window reset (Unix timestamp)	1700000000
Retry-After	Seconds to wait (on 429)	60

Exercise 7.3 — Exponential Backoff

Platform: Windows 10/11 · **PowerShell** for API · `$env:VAR · curl.exe`

```
def call_with_retry(url, max_retries=5, base_delay=1.0):
    for attempt in range(max_retries):
        response = requests.get(url, auth=AUTH)
        if response.status_code == 200:
            return response.json()
        if 400 <= response.status_code < 500:
            return None # Don't retry client errors
        if response.status_code in [429, 500, 502, 503, 504]:
            delay = int(response.headers.get('Retry-After',
                                             min(base_delay * (2 ** attempt), 60)))
            time.sleep(delay)
    return None
```

Exercise 7.3 — Rate Limiter & Client

Platform: Windows 10/11 · Prompts → **Agent panel** `Ctrl+L` · Diffs → **Editor**

Monitor headers: warn when `X-RateLimit-Remaining` < 10% of limit

Token bucket rate limiter: space requests evenly across the minute window

CursorAPIClient: combines rate limiting, retries on 429/5xx, timeout handling, and typed methods like `get_models()` and `create_agent()`

Success Criteria: Backoff · header monitoring · rate limiter · robust client class

Lesson 7.4

ETag Caching

Concept · 8 min · Exercise · 10 min

What Are ETags?

ETags are unique identifiers for API response versions.

- 1. Send `If-None-Match` header with previous ETag
- 2. Server returns `304 Not Modified` if unchanged
- 3. No data transfer, no rate limit consumption

ETag Flow

```
Request 1: GET /v1/analytics/usage
→ 200 OK, ETag: "abc123", Body: { ... data ... }

Request 2: GET with If-None-Match: "abc123"
→ 304 Not Modified (unchanged) OR
→ 200 OK, ETag: "def456" (updated data)
```

Endpoints Supporting ETags

Endpoint	ETag Support	Cache Freshness
<code>/v1/models</code>	✔ Yes	Changes rarely
<code>/v1/admin/members</code>	✔ Yes	Changes occasionally
<code>/v1/agents/{id}</code>	✔ Yes	Changes during run

Endpoint	ETag Support	Cache Freshness
/v1/analytics/usage	✔ Yes	Daily changes
/v1/agents (list)	⚠ Partial	Changes frequently

Exercise 7.4 — Basic ETag Usage

Platform: Windows 10/11 · **PowerShell** for API · `$env:VAR` · `curl.exe`

```
def get_with_etag(url, previous_etag=None):
    headers = {'If-None-Match': previous_etag} if previous_etag else {}
    response = requests.get(url, auth=AUTH, headers=headers)

    if response.status_code == 304:
        return None, response.headers.get('ETag') # Use cached data
    if response.status_code == 200:
        return response.json(), response.headers.get('ETag')
```

Exercise 7.4 — ETagCache & CachedClient

Platform: Windows 10/11 · Prompts → **Agent panel** `Ctrl+L` · Diffs → **Editor**

ETagCache: persistent pickle-based cache keyed by URL hash

CachedCursorClient:

- Check local cache → send `If-None-Match`
- On 304 → return cached data (Cache HIT)
- On 200 → update cache (Cache MISS)

Batch analytics: fetch 30 days of usage — unchanged days return 304 instantly

Success Criteria: Basic ETag request · persistent cache · analytics workload caching

Lesson 7.5

Listing Available Models

Concept · 4 min · Exercise · 6 min

The Models Endpoint

```
GET /v1/models
```

Response includes:

- Model ID · Display name · Context window size
- Pricing (input/output per 1M tokens)
- Capabilities (vision, tool calling, etc.)

"Simplest API call — perfect for verifying your API key works."

Exercise 7.5 — Steps 1–2

Platform: Windows 10/11 · **PowerShell** for API · `$env:VAR · curl.exe`

Step 1: List with curl: **Terminal: PowerShell** — `Ctrl+`` in Cursor

```
curl -s -u "$CURSOR_USER_API_KEY:" \
  https://api.cursor.com/v1/models \
  | jq '.data[] | {id: .id, context: .context_window, input_price:
    .pricing.input}'
```

Exercise 7.5 — Steps 1–2 (Part 2)

Step 2: Format with Python tabulate — Model ID, Context, Input/Output Price, Vision support **Terminal: PowerShell** — `python script.py`

Exercise 7.5 — Steps 3–4

Platform: Windows 10/11 · **PowerShell** for API · `$env:VAR · curl.exe`

Step 3: Filter models: **Terminal: PowerShell** — unless step notes Git Bash or WSL

```
# Models with 100k+ context
large_context = [m for m in models if m.get('context_window', 0) >= 100000]

# Cheapest by input price
cheapest = sorted(models, key=lambda x: x['pricing']['input'][:5])
```

Exercise 7.5 — Steps 3–4 (Part 2)

Step 4: Model selection helper: **Terminal: PowerShell** — unless step notes Git Bash or WSL


```
select_model("code_review", "balanced") # → claude-4.6-sonnet
select_model("simple_fix", "low")         # → gpt-5-mini
select_model("frontend_ui", "high")      # → gemini-3.1-pro
```

Module Summary

Lesson	Topic	Key Skill
7.1	API Landscape	API selection
7.2	Authentication	Key management
7.3	Rate Limits & Errors	Robust clients
7.4	ETag Caching	Efficient queries
7.5	Listing Models	Auth smoke-test

Quick Reference Card

```
BASE URL: https://api.cursor.com/v1

AUTH:  -u "api_key:" (curl) | Bearer token | OpenAI SDK base_url

ENDPOINTS:
  GET  /models           List available models
  POST /agents           Create cloud agent
  GET  /agents/{id}      Get agent status
  GET  /admin/members    List team members
  GET  /admin/analytics/usage Usage data

ERRORS:  429/5xx → retry with backoff | 4xx → fix request
CACHE:   If-None-Match → handle 304 Not Modified
```

Cloud Agents API and Webhooks

Module 8 · Day 2 (Hands-On)

Cursor Training Program · ~60 min

Module Overview

Aspect	Details
Duration	~60 minutes
Format	Hands-on exercise
Prerequisites	User API key (Module 7), Python 3.8+, ngrok installed, GitHub repository
Module Goal	Programmatically create, stream, and manage Cloud Agents, and set up webhook notifications

Learning Objectives

By the end of this module, participants will be able to:

- Create a Cloud Agent programmatically using the API
- Stream agent responses in real-time using SSE with resume support
- List and download artifacts from a completed agent
- Create a webhook endpoint with HMAC verification
- Test webhooks locally using ngrok
- Build an end-to-end automated agent workflow

Agenda

Lesson	Topic	Time
8.1	Creating a Cloud Agent Programmatically	15 min
8.2	Streaming Agent Responses (SSE)	15 min
8.3	Listing and Downloading Artifacts	15 min
8.4	Creating a Webhook Endpoint	15 min
8.5	Testing Webhooks Locally with ngrok	13 min
8.6	End-to-End Automated Agent Workflow	17 min

Lesson 8.1

Creating a Cloud Agent Programmatically

Concept · 5 min · Exercise · 10 min

Agent + Runs

Concept	Description
Agent	Durable entity with conversation history and workspace state

Concept	Description
Run	Single execution (one prompt/response cycle)

Key endpoint: `POST /v1/agents`

Request Fields

Required:

Field	Example
<code>prompt.text</code>	"Add a README.md file"
<code>repos[].url</code>	"https://github.com/org/repo"

Optional: `autoCreatePR` · `model.id` · `webhookUrl` · `webhookSecret`

Windows Exercise Environment

All exercises in this module assume **Windows 10/11** with Cursor installed.

Terminal	Use when	Open in Cursor
PowerShell	Default — Python, Git, <code>curl.exe</code> , npm, Cursor CLI (<code>agent</code>)	<code>Ctrl+`</code> → PowerShell
Git Bash	Bash syntax, <code>export VAR=...</code> , shell scripts ending in <code>.sh</code>	Terminal menu → Git Bash
Command Prompt	Legacy <code>.bat</code> files only	Terminal menu → Command Prompt
Ubuntu (WSL)	Linux-only tools or native bash without Git Bash	Terminal menu → Ubuntu (WSL)

Cursor Agent panel (`Ctrl+L`) is for natural-language prompts — not a shell.

Set default profile: Settings → `terminal.integrated.defaultProfile.windows` → **PowerShell**

Exercise 8.1 — Create with curl

Platform: Windows 10/11 · **PowerShell** for API · `$env:VAR` · `curl.exe`

Step 1 — set API key · **Terminal: PowerShell**

```
$env:CORSOR_USER_API_KEY = "cursor_xxxxxxxxxxxxx"
```

Step 2 — create agent · Terminal: PowerShell

```
curl.exe -X POST https://api.cursor.com/v1/agents `
  -u "$($env:CURL_USER_API_KEY):" `
  -H "Content-Type: application/json" `
  -d '{"prompt":{"text":"Add a README.md file with setup instructions"},"repos":
[{"url":"https://github.com/YOUR_ORG/YOUR_REPO","startingRef":"main"}],"autoCreate
PR":true}' `
  | ConvertFrom-Json
```

Terminal (alternative): Git Bash / WSL — bash block below.

```
export CURL_USER_API_KEY="cursor_XXXXXXXXXXXXX"
curl -X POST https://api.cursor.com/v1/agents -u "$CURL_USER_API_KEY:" -H
"Content-Type: application/json" -d '{"prompt":{"text":"Add a README.md file
with setup instructions"},"repos":
[{"url":"https://github.com/YOUR_ORG/YOUR_REPO","startingRef":"main"}],"autoCreate
PR":true}' | jq '.'
```

Exercise 8.1 — Capture IDs

Platform: Windows 10/11 · **PowerShell** for API · `$env:VAR` · `curl.exe`

Step 1: Save the JSON from the create-agent call — **Terminal: PowerShell**

```
$response = curl.exe ... | ConvertFrom-Json # reuse create-agent command
$env:AGENT_ID = $response.agent.id
$env:RUN_ID = $response.run.id
Write-Host "Agent ID: $($env:AGENT_ID)"
Write-Host "Dashboard: https://cursor.com/agents/$($env:AGENT_ID)"
```

Exercise 8.1 — Capture IDs (Part 2)

Step 2: Optional model override in create payload — **Where:** edit JSON before POST (any terminal)

Create with specific model: `"model": {"id": "claude-4.7-opus"}`

Exercise 8.1 — Python Helper

Platform: Windows 10/11 · **PowerShell** for API · `$env:VAR` · `curl.exe`

```
def create_agent(prompt, repo_url, auto_create_pr=False, model=None):
    payload = {
```

```
    "prompt": {"text": prompt},
    "repos": [{"url": repo_url}],
    "autoCreatePR": auto_create_pr
  }
  if model:
    payload["model"] = {"id": model}
  response = requests.post(f"{BASE_URL}/agents", auth=AUTH, json=payload)
  data = response.json()
  return data["agent"]["id"], data["run"]["id"]
```

Success Criteria: Agent created · IDs captured · appears in dashboard · Python function works

Lesson 8.2

Streaming Agent Responses (SSE)

Concept · 5 min · Exercise · 10 min

SSE Event Types

Event	When It Happens	Data Example
status	Run status changes	{"status": "RUNNING"}
assistant	Agent speaks	{"text": "I'll read the file..."}
thinking	Agent is reasoning	{"text": "Let me consider..."}
tool_call	Agent uses a tool	{"name": "read_file", "status": "started"}
result	Run completes	{"status": "FINISHED"}
error	Something went wrong	{"message": "..."}
done	Stream ends	{}

Resume Support

SSE streams support the **Last-Event-ID** header — if your connection drops, resume from the last received event.

Exercise 8.2 — Stream with curl

Platform: Windows 10/11 · **PowerShell** for API · `$env:VAR` · `curl.exe`

Terminal: PowerShell

```
curl.exe -N -u "$($env:CORSOR_USER_API_KEY):" `
-H "Accept: text/event-stream" `
"https://api.cursor.com/v1/agents/$env:AGENT_ID/runs/$env:RUN_ID/stream"
```

Set IDs first: `$env:AGENT_ID = "..."` · `$env:RUN_ID = "..."`

Terminal (alternative): Git Bash / WSL — bash `curl -N` block above.

Parse lines starting with `event:` and `data:` — print assistant text, tool calls, and result status.

Exercise 8.2 — Python SSE Client

Platform: Windows 10/11 · **PowerShell** for API · `$env:VAR` · `curl.exe`

```
def stream_agent_response(agent_id, run_id, on_event=None):
    url = f"{BASE_URL}/agents/{agent_id}/runs/{run_id}/stream"
    response = requests.get(url, auth=AUTH, stream=True)
    for line in response.iter_lines():
        if line.startswith(b'event:'):
            current_event = line[6:].strip().decode()
        elif line.startswith(b'data:'):
            data = json.loads(line[5:].strip())
            if on_event:
                on_event(current_event, data)
```

Exercise 8.2 — ResumableSSEClient

Platform: Windows 10/11 · Prompts → **Agent panel** `Ctrl+L` · Diffs → **Editor**

Track `last_event_id` from `id:` lines → send as `Last-Event-ID` header on reconnect

Also: `stream_to_file()` saves full SSE log for later review

Success Criteria: Stream connected · received events · Python client works · resume implemented

Lesson 8.3

Listing and Downloading Artifacts

Concept · 5 min · Exercise · 10 min

Key Endpoints

Endpoint	Method	Purpose
/v1/agents/{id}/artifacts	GET	List all artifacts
/v1/agents/{id}/artifacts/download	GET	Get presigned URL for download

Important: Download URLs expire after **15 minutes**.

Exercise 8.3 — Wait & List

Platform: Windows 10/11 · **PowerShell** for API · `$env:VAR · curl.exe`

```
def wait_for_completion(agent_id, timeout=300, poll_interval=5):
    while time.time() - start < timeout:
        status = get_agent_status(agent_id).get('status')
        if status == 'FINISHED': return True
        elif status == 'ERROR': return False
        time.sleep(poll_interval)

def list_artifacts(agent_id):
    response = requests.get(f"{BASE_URL}/agents/{agent_id}/artifacts", auth=AUTH)
    return response.json().get('items', [])
```

Exercise 8.3 — Download

Platform: Windows 10/11 · **PowerShell** for API · `$env:VAR · curl.exe`

Single artifact:

```
response = requests.get(
    f"{BASE_URL}/agents/{agent_id}/artifacts/download",
    auth=AUTH, params={"path": artifact_path}
)
download_url = response.json().get('url')
# curl download_url → save to disk
```

All artifacts: loop items, create subdirs, download each via presigned URL

Exercise 8.3 — CI Integration

Platform: Windows 10/11 · **PowerShell** for API · `$env:VAR · curl.exe`

```
def process_test_results(agent_id):
    wait_for_completion(agent_id, timeout=600)
```

```
download_artifact(agent_id, "artifacts/junit.xml", "test_results.xml")
# Parse XML → exit 1 if failures/errors, else exit 0
```

Success Criteria: Listed artifacts · downloaded single + all · CI workflow integration

Lesson 8.4

Creating a Webhook Endpoint

Concept · 5 min · Exercise · 10 min

Webhook Headers

Header	Description
X-Webhook-Signature	HMAC-SHA256 signature for verification
X-Webhook-ID	Unique delivery ID
X-Webhook-Event	Event type (<code>statusChange</code>)

Webhook Payload

```
{
  "event": "statusChange",
  "id": "agent_abc123",
  "status": "FINISHED",
  "source": {"repository": "https://github.com/your-org/your-repo"},
  "target": {"url": "...", "prUrl": "https://github.com/.../pull/123"},
  "summary": "Added README.md and fixed tests"
}
```

Exercise 8.4 — HMAC Verification

Platform: Windows 10/11 · **PowerShell** for API · `$env:VAR` · `curl.exe`

```
def verify_signature(raw_body, signature_header):
    received = signature_header[7:] # strip "sha256="
    expected = hmac.new(
        WEBHOOK_SECRET.encode(), raw_body, hashlib.sha256
    ).hexdigest()
    return hmac.compare_digest(expected, received)
```


Flask route: verify signature → parse payload → handle FINISHED/ERROR

Exercise 8.4 — Configure Agent

Platform: Windows 10/11 · **PowerShell** for API · `$env:VAR` · `curl.exe`

```
curl -X POST https://api.cursor.com/v1/agents \
-u "$CURSOR_USER_API_KEY:" \
-H "Content-Type: application/json" \
-d '{
  "prompt": {"text": "Add a CONTRIBUTING.md file"},
  "repos": [{"url": "https://github.com/YOUR_ORG/YOUR_REPO"}],
  "webhookUrl": "https://your-domain.com/webhook/cursor",
  "webhookSecret": "your-secret-here",
  "autoCreatePR": true
}'
```

PowerShell (Windows): Same steps in **PowerShell** — use `$env:NAME = "value"` instead of `export`, and `curl.exe` instead of `curl`.

Success Criteria: Server running · signature verified · payload parsed · agent configured

Lesson 8.5

Testing Webhooks Locally with ngrok

Concept · 5 min · Exercise · 8 min

What Is ngrok?

Creates a secure tunnel from a public URL to your local server.

- Test webhooks without deploying
 - Debug locally · Demo to stakeholders
-

Exercise 8.5 — Steps 1–3

Platform: Windows 10/11 · **PowerShell** for API · `$env:VAR` · `curl.exe`

Step 1: Start tunnel: **Terminal: PowerShell** — unless step notes Git Bash or WSL

```
ngrok http 5000
# Forwarding: https://abc123.ngrok.io -> http://localhost:5000
```

Exercise 8.5 — Steps 1–3 (Part 2)

Step 2: Copy HTTPS URL **Terminal: PowerShell** — unless step notes Git Bash or WSL

Exercise 8.5 — Steps 1–3 (Part 3)

Step 3: Create agent with ngrok URL: **Terminal: PowerShell** — **Ctrl+`** in Cursor

```
curl -X POST https://api.cursor.com/v1/agents ... \
  -d '{"webhookUrl": "https://abc123.ngrok.io/webhook/cursor", ...}'
```

Exercise 8.5 — Inspect & Replay

Platform: Windows 10/11 · Agent → **Ctrl+L** · Shell → **PowerShell** · Browser for dashboards

Step 4: Inspect requests at <http://127.0.0.1:4040> **Terminal: PowerShell** — unless step notes Git Bash or WSL

Step 5: Replay failed webhooks (ngrok premium) — inspect raw body and headers **Terminal: Git Bash** or **Ubuntu (WSL)** — bash syntax required

Success Criteria: Tunnel established · webhook received · signature verified · inspected in ngrok UI

Lesson 8.6

End-to-End Automated Agent Workflow

Concept · 5 min · Exercise · 12 min

The Capstone Integration

Combine everything into `automated_workflow.py`:

1. **Create agent** (with optional webhook URL)
 2. **Wait for completion** (webhook or polling)
 3. **Download artifacts**
 4. **Process results** (CI exit codes, notifications)
-

Workflow Architecture

```
create_agent() → wait (webhook OR polling) → download_artifacts()
      ↑               ↑
```

Flask webhook server completion_event.set()
(background thread) on FINISHED status

Run the Workflow

```
export CURSOR_USER_API_KEY="cursor_XXXXXXXXXXXX"

python automated_workflow.py \
  --repo "https://github.com/YOUR_ORG/YOUR_REPO" \
  --prompt "Add a comprehensive README.md with setup and API docs" \
  --output "./outputs"

# Polling only (no webhook):
python automated_workflow.py --repo "..." --prompt "..." --no-webhook
```

Workflow Output

 CLOUD AGENT AUTOMATED WORKFLOW

 Creating agent... Agent ID: agt_abc123

 Waiting for completion...

 Webhook received: Agent agt_abc123 completed

 Downloaded 3 artifacts to agent_outputs/

 WORKFLOW COMPLETE

Success Criteria: Creates agent · waits (webhook/polling) · downloads artifacts · end-to-end run

Module Summary

Lesson	Topic	Key Skill
8.1	Creating a Cloud Agent	Programmatic agent launch
8.2	Streaming Agent Responses	SSE with resume support
8.3	Listing and Downloading Artifacts	CI pipeline integration
8.4	Creating a Webhook Endpoint	HMAC verification
8.5	Testing Webhooks with ngrok	Local tunnel debugging
8.6	End-to-End Workflow	Complete automation

Quick Reference Card

```
ENDPOINTS:
  POST /v1/agents                Create agent
  GET  /v1/agents/{id}           Get status
  GET  /v1/agents/{id}/runs/{id}/stream SSE stream
  GET  /v1/agents/{id}/artifacts List artifacts
  GET  /v1/agents/{id}/artifacts/download Download

WEBHOOK:  X-Webhook-Signature (HMAC-SHA256) | X-Webhook-Event

SSE:  status · assistant · thinking · tool_call · result · error · done

NGROK:  ngrok http 5000 | inspect at http://127.0.0.1:4040
```

Admin and Analytics APIs

Module 9 · Day 2 (Hands-On + Demonstrations)

Cursor Training Program · ~75 min

Module Overview

Aspect	Details
Duration	~75 minutes
Format	Hands-on exercise + demonstrations
Prerequisites	Admin API key (not User key), Python 3.8+, Modules 7–8 completed
Module Goal	Master team management, usage analytics, cost governance, and safe admin operations

Learning Objectives

By the end of this module, participants will be able to:

- List and manage team members programmatically
- Retrieve daily usage data for cost tracking and reporting
- Set user spend limits for budget governance
- Analyze model usage for cost optimization insights
- Track daily active users for leadership reporting
- Build responsible leaderboards without privacy violations
- Analyze conversation intent and complexity (demonstration)
- Safely remove team members with proper patterns (demonstration)

Agenda

Lesson	Topic	Time	Type
9.1	Listing Team Members	8 min	Exercise
9.2	Daily Usage Data	10 min	Exercise
9.3	Setting User Spend Limits	8 min	Exercise
9.4	Model Usage Analytics	8 min	Exercise
9.5	Daily Active Users	6 min	Exercise
9.6	Leaderboards	6 min	Exercise
9.7	Conversation Insights	6 min	Demo
9.8	Destructive Admin Operations	6 min	Demo

Lesson 9.1

Listing Team Members

Concept · 5 min · Exercise · 8 min

User vs. Admin API Key

Aspect	User API Key	Admin API Key
Scope	Your user only	Entire organization
Can list members	✗ No	✓ Yes
Can view others' usage	✗ No	✓ Yes
Can modify policies	✗ No	✓ Yes
Format	cursor_xxx...	cursor_admin_xxx...

Key endpoint: `GET /v1/admin/members`

Windows Exercise Environment

All exercises in this module assume **Windows 10/11** with Cursor installed.

Terminal	Use when	Open in Cursor
PowerShell	Default — Python, Git, <code>curl.exe</code> , npm, Cursor CLI (<code>agent</code>)	<code>Ctrl+`</code> → PowerShell
Git Bash	Bash syntax, <code>export VAR=...</code> , shell scripts ending in <code>.sh</code>	Terminal menu → Git Bash

Terminal	Use when	Open in Cursor
Command Prompt	Legacy <code>.bat</code> files only	Terminal menu → Command Prompt
Ubuntu (WSL)	Linux-only tools or native bash without Git Bash	Terminal menu → Ubuntu (WSL)

Cursor Agent panel (`Ctrl+L`) is for natural-language prompts — not a shell.

Set default profile: Settings → `terminal.integrated.defaultProfile.windows` → **PowerShell**

Exercise 9.1 — Setup & List

Platform: Windows 10/11 · **PowerShell** for API · `$env:VAR · curl.exe`

```
export CURSOR_ADMIN_API_KEY="cursor_admin_XXXXXXXXXXXX"

# Verify admin access
curl -s -u "$CURSOR_ADMIN_API_KEY:" \
  https://api.cursor.com/v1/admin/organization | jq '.'

# List all team members
curl -s -u "$CURSOR_ADMIN_API_KEY:" \
  "https://api.cursor.com/v1/admin/members" | jq '.'
```

PowerShell (Windows): Same steps in **PowerShell** — use `$env:NAME = "value"` instead of `export`, and `curl.exe` instead of `curl`.

Exercise 9.1 — Pagination & Export

Platform: Windows 10/11 · **PowerShell** for API · `$env:VAR · curl.exe`

Pagination:

```
curl -s -u "$CURSOR_ADMIN_API_KEY:" \
  "https://api.cursor.com/v1/admin/members?limit=10&offset=0"
```

PowerShell (Windows): Same steps in **PowerShell** — use `$env:NAME = "value"` instead of `export`, and `curl.exe` instead of `curl`.

Python: loop with offset until empty → export to `team_roster.csv` (email, role, status, joined, lastActiveAt)

Helper: `get_user_id_by_email(email)` for downstream admin calls

Success Criteria: Authenticated · listed members · handled pagination · exported CSV

Lesson 9.2

Daily Usage Data

Concept · 5 min · Exercise · 10 min

Key Endpoint

GET /v1/admin/analytics/usage/daily

Returns:

- Cost per day · Input/output token counts
- Active users per day · Breakdown by user and model (optional)

"Finance asks: 'What did we spend yesterday?' Engineering leads ask: 'Who's using what?'"

Exercise 9.2 — Weekly Usage

Platform: Windows 10/11 · **PowerShell** for API · `$env:VAR` · `curl.exe`

```
END=$(date +%Y-%m-%d)
START=$(date -d "7 days ago" +%Y-%m-%d)

curl -s -u "$CURSOR_ADMIN_API_KEY:" \
  "https://api.cursor.com/v1/admin/analytics/usage/daily?
startDate=$START&endDate=$END" \
  | jq '.daily[] | {date: .date, cost: .cost, tokens: .totalTokens, users:
.activeUsers}'
```

PowerShell (Windows): Same steps in **PowerShell** — use `$env:NAME = "value"` instead of `export`, and `curl.exe` instead of `curl`.

Exercise 9.2 — Cost Report

Platform: Windows 10/11 · Prompts → **Agent panel** `Ctrl+L` · **Diff**s → **Editor**

Python `generate_cost_report()` for last 30 days:

- Total cost · total tokens · average daily cost/users
- Week-over-week change · top 5 costliest days
- Daily breakdown table (last 14 days)
- Budget alerts at \$300 / \$500 thresholds

Success Criteria: Retrieved date range · calculated trends · generated readable report

Lesson 9.3

Setting User Spend Limits

Concept · 5 min · Exercise · 8 min

Key Endpoint

PATCH /v1/admin/policies/users/{userId}/limits

Action	Behavior
alert	Send notification but allow usage
block	Prevent any further requests for the month

Exercise 9.3 — Set Limits

Platform: Windows 10/11 · **PowerShell** for API · `$env:VAR` · `curl.exe`

```
USER_ID=$(curl -s -u "$CURSOR_ADMIN_API_KEY:" \
  "https://api.cursor.com/v1/admin/members?email=developer@company.com" \
  | jq -r '.members[0].id')

curl -X PATCH ".../policies/users/$USER_ID/limits" \
  -u "$CURSOR_ADMIN_API_KEY:" \
  -d '{"monthlyLimit": 50.00, "exceedanceAction": "block"}'
```

PowerShell (Windows): Same steps in **PowerShell** — use `$env:NAME = "value"` instead of `export`, and `curl.exe` instead of `curl`.

Check current limit: GET .../policies/users/{userId}/limits

Exercise 9.3 — Bulk Limits

Platform: Windows 10/11 · Prompts → **Agent panel** `Ctrl+L` · Diffs → **Editor**

CSV bulk set: email, monthly_limit, action

```
intern@company.com,20,block
contractor@company.com,50,alert
lead@company.com,200,alert
```

Find heavy users: query `/analytics/usage/users` for current month → filter cost > threshold

Success Criteria: Retrieved user ID · set limit · verified · bulk setting implemented

Lesson 9.4

Model Usage Analytics

Concept · 5 min · Exercise · 8 min

Key Endpoint

GET /v1/admin/analytics/usage/models

"Which models are actually being used? Is Opus worth the cost? Should you train people on cheaper alternatives?"

Exercise 9.4 — Model Breakdown

Platform: Windows 10/11 · **PowerShell** for API · `$env:VAR` · `curl.exe`

```
curl -s -u "$CURSOR_ADMIN_API_KEY:" \
  ".../analytics/usage/models?startDate=$START&endDate=$END" \
  | jq '.models[] | {model: .modelId, cost: .cost, requests: .requestCount}'

# Find Opus overuse per user
curl -s -u "$CURSOR_ADMIN_API_KEY:" \
  ".../analytics/usage/users?startDate=$START&endDate=$END" \
  | jq '.users[] | select(.modelBreakdown."claude-4.7-opus" != null)'
```

PowerShell (Windows): Same steps in **PowerShell** — use `$env:NAME = "value"` instead of `export`, and `curl.exe` instead of `curl`.

Exercise 9.4 — Optimization Report

Platform: Windows 10/11 · Prompts → **Agent panel** `Ctrl+L` · Diffs → **Editor**

`generate_optimization_report()`:

- Model cost breakdown (% of total)
- Users on Claude Opus → suggest Sonnet for non-critical tasks
- High Sonnet usage → suggest GPT-5.3 Codex (40% savings)
- Estimated monthly savings if guidelines applied

Success Criteria: Retrieved model breakdown · identified expensive users · generated recommendations

Lesson 9.5

Daily Active Users (DAU)

Concept · 4 min · Exercise · 6 min

Why DAU Matters

- Track adoption after rollout
- Identify unused licenses for reallocation
- Measure impact of training sessions
- Justify renewal and expansion

Source: `activeUsers` field from `/admin/analytics/usage/daily`

Exercise 9.5 — DAU Report

Platform: Windows 10/11 · **PowerShell** for API · `$env:VAR` · `curl.exe`

```
curl -s -u "$CURSOR_ADMIN_API_KEY:" \  
  ".../analytics/usage/daily?startDate=$START&endDate=$END" \  
  | jq '{avg_weekly: ([.daily[-7:] | .[].activeUsers] | add / length),  
        peak: ([.daily[] | .activeUsers] | max)}'
```

PowerShell (Windows): Same steps in **PowerShell** — use `$env:NAME = "value"` instead of `export`, and `curl.exe` instead of `curl`.

Python leadership report:

- Average/median/peak DAU · adoption rate (% of team)
- WoW growth rate · weekly averages
- Health assessment: >80% excellent · >50% good · <30% investigate

Success Criteria: Calculated DAU · adoption metrics · leadership-ready report

Lesson 9.6

Leaderboards

Concept · 5 min · Exercise · 6 min

Responsible Leaderboard Principles

Principle	Implementation
Anonymize	Roles or anonymized names, not full emails
Focus on positive metrics	Show savings, not spending
Opt-in only	Allow users to choose public visibility
Include context	Show team size, role differences

Exercise 9.6 — Three Leaderboards

Platform: Windows 10/11 · **PowerShell** for API · `$env:VAR` · `curl.exe`

- 1. **Engagement leaderboard** — rank by request count (anonymized emails)
- 2. **Efficiency leaderboard** — tokens per dollar spent
- 3. **Savings leaderboard** — users who saved by choosing efficient models over Opus

```
def anonymize_email(email):  
    local = email.split('@')[0]  
    return local[:2] + "... " + local[-2:]
```

Success Criteria: Anonymized · efficiency-focused · savings-focused leaderboards

Lesson 9.7

Conversation Insights

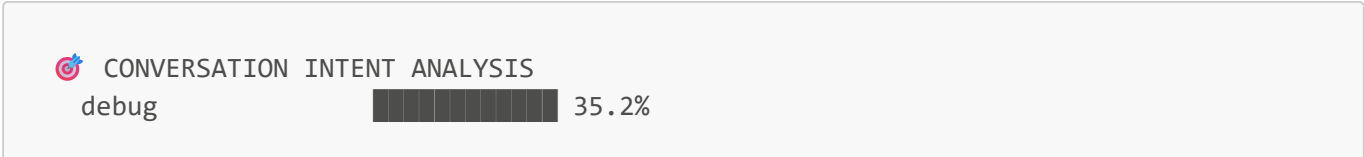
Concept · 6 min · Demonstration

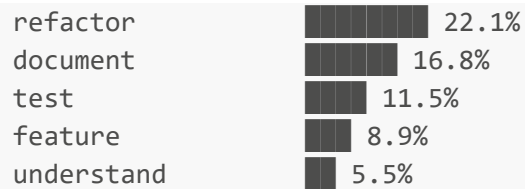
What Conversation Insights Reveal

- Simple questions vs. complex refactors?
- Most common task categories
- Where users get stuck
- Which models perform best for which task types

Endpoint: `GET /v1/admin/analytics/conversations` (may require Enterprise plan)

Demo: Intent Analysis





Demo: Complexity & Categories

Complexity distribution:

- simple 45% · moderate 33% · complex 15% · architectural 7%

Category analysis:

- backend 40% · frontend 29% · database 15% · devops 10% · security 6%

Stuck patterns: conversations >5 min with **success: false** → suggest training/docs

Success Criteria: Understood capabilities · intent/complexity/category tracking · stuck patterns

Lesson 9.8

Destructive Admin Operations

Concept · 6 min · Demonstration

Safe Removal Playbook

1. **Audit first** — active agents, runs, API keys
 2. **Soft delete** — deactivate (no new agents; existing continue)
 3. **Transfer ownership** — critical agents, webhooks
 4. **Log everything** — compliance audit trail
 5. **Confirm before hard delete** — GDPR/security only
-

Demo: SafeRemovalDemo Workflow

```
Step 1: find_user(email) → user_id
Step 2: audit_resources() → agents, runs
Step 3: deactivate() → status: inactive
Step 4: transfer_resources(new_owner_email)
Step 5: hard_delete() → permanent (rare)
→ generate_audit_report()
```

Bulk deactivation: find users inactive 90+ days → review → notify → deactivate

Success Criteria: 5-step pattern · audit-first · soft vs hard delete · resource transfer

Module Summary

Lesson	Topic	Type
9.1	Listing Team Members	Exercise
9.2	Daily Usage Data	Exercise
9.3	Setting User Spend Limits	Exercise
9.4	Model Usage Analytics	Exercise
9.5	Daily Active Users	Exercise
9.6	Leaderboards	Exercise
9.7	Conversation Insights	Demo
9.8	Destructive Operations	Demo

Quick Reference Card

ENDPOINTS:

GET	/admin/members	List members
GET	/admin/analytics/usage/daily	Daily usage
GET	/admin/analytics/usage/models	Model usage
GET	/admin/analytics/usage/users	Per-user usage
PATCH	/admin/policies/users/{id}/limits	Set spend limits
GET	/admin/members/{id}/resources	User resources

LEADERBOARDS: Anonymize · positive metrics · opt-in · role context

REMOVAL: Audit → Deactivate → Transfer → Log → Confirm delete

AI Code Tracking and Reporting

Module 10 · Day 2 (Hands-On + Take-Home Project)

Cursor Training Program · ~20 min + take-home

Module Overview

Aspect	Details
Duration	~20 minutes (plus take-home project)

Aspect	Details
Format	Hands-on exercise + take-home project
Prerequisites	Admin API key, Git repository access, Modules 8–9 completed
Module Goal	Track AI vs. human contributions, export metrics to BI tools, build compliance dashboards

Learning Objectives

By the end of this module, participants will be able to:

- Attribute AI vs. human contributions per commit
- Stream metrics to BI tools via CSV export
- Access granular AI change events for compliance
- Build a complete reporting dashboard combining all data sources

Agenda

Lesson	Topic	Time
10.1	AI Commit Metrics	8 min
10.2	Bulk Export via CSV Streaming	7 min
10.3	Granular AI Change Events	7 min
10.4	Reporting Dashboard Architecture	4 min + take-home

Lesson 10.1

AI Commit Metrics

Concept · 3 min · Exercise · 5 min

Key Endpoint

GET /v1/admin/analytics/commits

What this measures:

- Lines added by AI vs. human
- Files modified by agent vs. manual
- Commit-level attribution · Per-developer breakdown

"The 'ROI of AI' metric — how much code was AI-generated vs. human-written."

Windows Exercise Environment

All exercises in this module assume **Windows 10/11** with Cursor installed.

Terminal	Use when	Open in Cursor
PowerShell	Default — Python, Git, <code>curl.exe</code> , npm, Cursor CLI (<code>agent</code>)	<code>Ctrl+`</code> → PowerShell
Git Bash	Bash syntax, <code>export VAR=...</code> , shell scripts ending in <code>.sh</code>	Terminal menu → Git Bash
Command Prompt	Legacy <code>.bat</code> files only	Terminal menu → Command Prompt
Ubuntu (WSL)	Linux-only tools or native bash without Git Bash	Terminal menu → Ubuntu (WSL)

Cursor Agent panel (`Ctrl+L`) is for natural-language prompts — not a shell.

Set default profile: Settings → `terminal.integrated.defaultProfile.windows` → **PowerShell**

Exercise 10.1 — Fetch Metrics

Platform: Windows 10/11 · **PowerShell** for API · `$env:VAR · curl.exe`

```
END=$(date +%Y-%m-%d)
START=$(date -d "30 days ago" +%Y-%m-%d)

curl -s -u "$CURSOR_ADMIN_API_KEY:" \
  ".../analytics/commits?
startDate=$START&endDate=$END&repo=https://github.com/YOUR_ORG/YOUR_REPO" \
  | jq .'
```

PowerShell (Windows): Same steps in **PowerShell** — use `$env:NAME = "value"` instead of `export`, and `curl.exe` instead of `curl`.

Exercise 10.1 — AI Contribution %

Platform: Windows 10/11 · **PowerShell** for API · `$env:VAR · curl.exe`

```
curl -s -u "$CURSOR_ADMIN_API_KEY:" \
  ".../analytics/commits?startDate=$START&endDate=$END" \
  | jq '{
    total_commits: .summary.totalCommits,
    ai_commits: .summary.aiAuthoredCommits,
    ai_percentage: (.summary.aiAuthoredCommits / .summary.totalCommits * 100),
    lines_saved: .summary.aiGeneratedLines
  }'
```

PowerShell (Windows): Same steps in **PowerShell** — use `$env:NAME = "value"` instead of `export`, and `curl.exe` instead of `curl`.

Exercise 10.1 — ROI Analysis

Platform: Windows 10/11 · Prompts → **Agent panel** `Ctrl+L` · Diffs → **Editor**

Python `calculate_ai_roi()`:

```
AI-generated lines vs. human-written lines (%)
Estimated time saved (10 lines/min assumption)
Estimated cost saved ($100/hr developer cost)
AI usage cost → Net ROI
```

`contributor_breakdown()` — AI %, AI lines, commits per developer

Success Criteria: Retrieved metrics · calculated AI % · generated ROI analysis

Lesson 10.2

Bulk Export via CSV Streaming

Concept · 3 min · Exercise · 4 min

Key Endpoint

`GET /v1/admin/analytics/export/csv` (streaming)

"Wire metrics into BI tools (Tableau, PowerBI, Looker, Metabase) without timeouts."

Export types: `commits` · `events` · `usage`

Exercise 10.2 — Stream to File

Platform: Windows 10/11 · **PowerShell** for API · `$env:VAR` · `curl.exe`

```
curl -N -u "$CURSOR_ADMIN_API_KEY:" \  
  ".../analytics/export/csv?startDate=$START&endDate=$END&type=commits" \  
  -o cursor_commits_export.csv  
  
head -10 cursor_commits_export.csv
```

PowerShell (Windows): Same steps in **PowerShell** — use `$env:NAME = "value"` instead of `export`, and `curl.exe` instead of `curl`.

Exercise 10.2 — BI Integration

Platform: Windows 10/11 · **PowerShell** for API · `$env:VAR` · `curl.exe`

Python `stream_to_dataframe()` → pandas DataFrame:

```
export_for_bi():  
  bi_commits.csv    # commit data  
  bi_events.csv     # event data  
  bi_usage.csv      # usage data
```

Upload to Metabase, PowerBI, or Tableau via CSV import

Success Criteria: Streamed CSV · loaded into DataFrame · created BI-ready files

Lesson 10.3

Granular AI Change Events

Concept · 3 min · Exercise · 4 min

Key Endpoint

`GET /v1/admin/analytics/events`

Tracks per event:

- File, line range, model used, timestamp
- User, accepted/rejected status

"Essential for SOC2, ISO, and internal audits."

Exercise 10.3 — Query Events

Platform: Windows 10/11 · **PowerShell** for API · `$env:VAR` · `curl.exe`

```
curl -s -u "$CURSOR_ADMIN_API_KEY:" \  
  ".../analytics/events?startDate=$START&endDate=$END&limit=100" \  
  | jq '.events[] | {user: .user.email, file: .filePath, model: .modelId, \  
    accepted: .accepted}'
```

PowerShell (Windows): Same steps in **PowerShell** — use `$env:NAME = "value"` instead of `export`, and `curl.exe` instead of `curl`.

Acceptance rate by model: group events → total vs. accepted per model

Exercise 10.3 — Compliance Report

Platform: Windows 10/11 · Prompts → **Agent panel** **Ctrl+L** · Diffs → **Editor**

`generate_compliance_report()` for last 90 days:

- Acceptance rate by model (table)
- Top 10 files with most AI changes (needs review)
- Export `compliance_export.csv` for auditors:
 - timestamp, user_email, model_id, file_path, line_start, line_end, accepted

Success Criteria: Retrieved events · calculated acceptance rates · compliance export

Lesson 10.4

Reporting Dashboard Architecture

Concept · 4 min · Take-Home Project

Dashboard Components

Component	Data Source	Purpose
Usage Overview	Usage API	Cost, tokens, active users
AI Contribution	Commits API	ROI, adoption metrics
Model Performance	Events API	Acceptance rates, efficiency
Team Activity	Members API	Onboarding, licensing
Compliance	Events + Audit	Audit trail, security

Take-Home: Streamlit Dashboard

Run with: `streamlit run cursor_dashboard.py`

5 panels:

1. **Executive Summary** — cost, DAU, AI %, team size
2. **Usage Analytics** — daily cost trend (Plotly line chart)
3. **AI Code Impact** — AI vs human bar chart + acceptance by model
4. **Team Management** — member table
5. **Compliance Export** — download events CSV

Project Deliverables

Deliverable	Description
Working dashboard	Streamlit, Metabase, or custom frontend
Documentation	Setup instructions and data source descriptions
One insight	Key finding from your team's data
Export script	Automated CSV export for compliance

Bonus: cost alerts · spend limit UI · model A/B comparison · cloud deployment

Module Summary

Lesson	Topic	Key Skill
10.1	AI Commit Metrics	ROI calculation
10.2	Bulk Export via CSV	BI integration
10.3	Granular Change Events	Compliance reporting
10.4	Dashboard Architecture	Complete dashboard

Quick Reference Card

ENDPOINTS:

GET /admin/analytics/commits

AI per commit

GET /admin/analytics/export/csv

Bulk CSV export

GET /admin/analytics/events

Granular events

KEY METRICS:

Adoption:

DAU / total team size

ROI:

(Hours saved × rate) - AI cost

Quality:

Acceptance rate by model

Compliance:

Complete change audit trail

DEPLOY: Streamlit · Metabase · PowerBI · Custom React

Course Complete

Cursor Training Program — Day 2

You can now:

- Create and manage Cloud Agents programmatically

- Stream agent responses with SSE · Webhooks with HMAC verification
- Administer teams and enforce spending policies
- Analyze usage, model adoption, and AI contribution
- Track AI vs. human code changes per commit
- Build reporting dashboards for leadership

Next steps: Complete take-home dashboard · set up scheduled exports · implement cost alerts

End of Module 10 — End of Day 2 Training Program